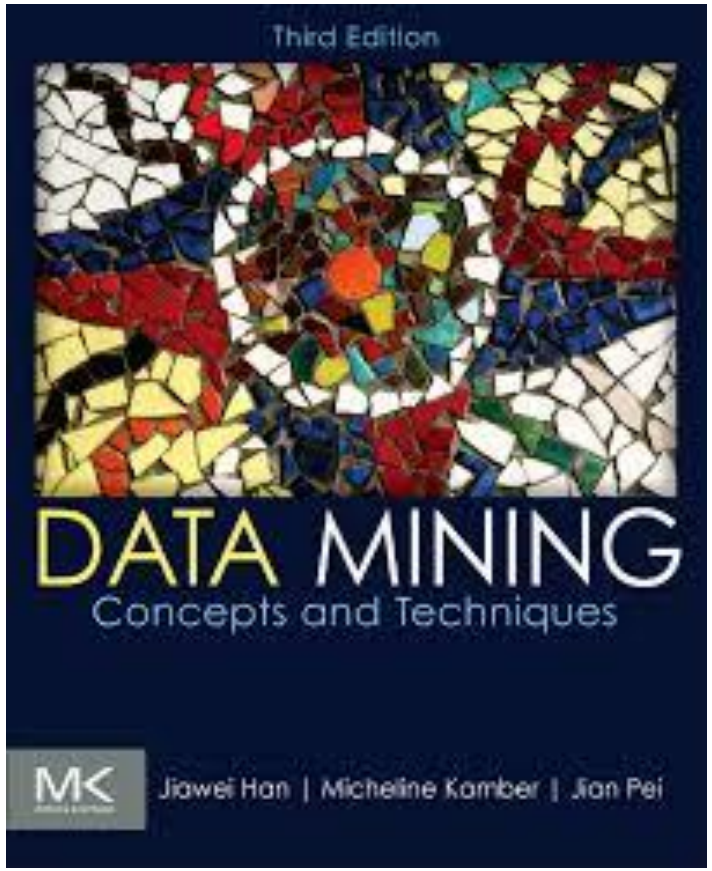
Machine Learning and Data Mining Basics

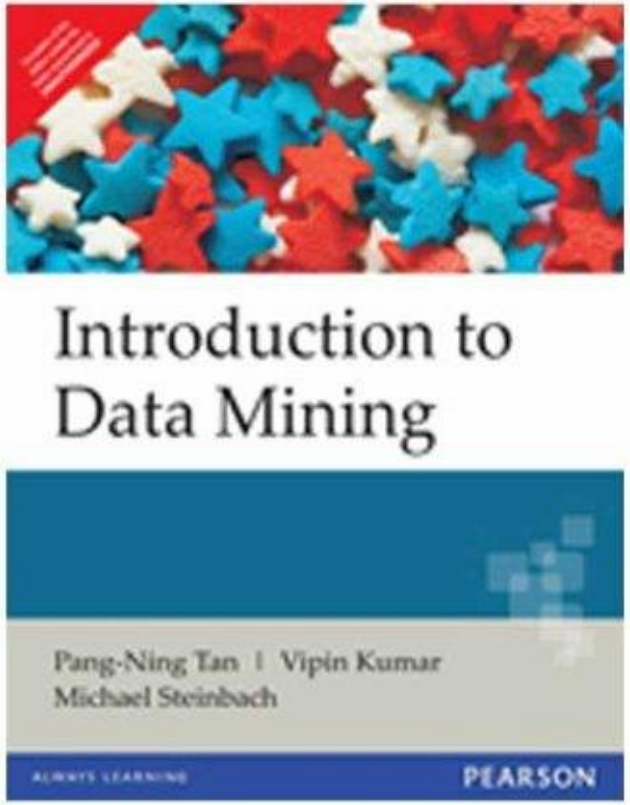
Course Goal

- **Differentiate Data Mining and Machine Learning** – Understand the scope, purpose, and key differences.
- **Understand Data Representation** – Learn how data is structured, formatted, and encoded for ML.
- **Master Data Preprocessing** – Handle missing data, scaling, normalization, and feature engineering.
- **Comprehend the Machine Learning Life Cycle** – From problem definition to model deployment.
- **Identify Types of Machine Learning** – Supervised, Semi-supervised, Unsupervised, and Reinforcement Learning.
- **Implement Core ML Algorithms** – Linear & logistic regression, decision trees, SVM, KNN, Naïve Bayes, etc.
- **Evaluate ML Models** – Metrics, validation, cross-validation, and performance improvement techniques.
- **Introduction to Deep Learning** – Basics of neural networks and their applications.
- **Develop Practical Thinking Skills** – Learn frameworks for problem-solving and common beginner pitfalls.

Course Materials and References

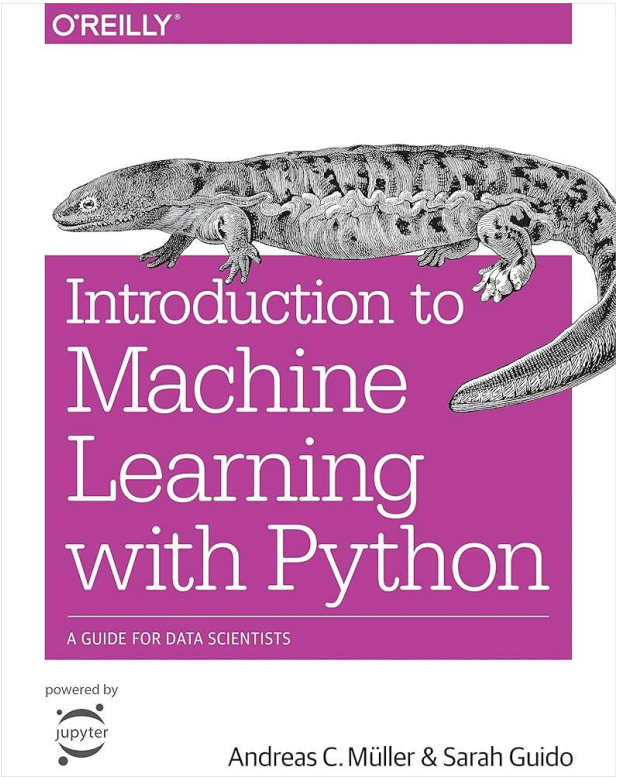


Data Mining Concepts and Techniques by Jiawei Han, Micheline Kamber, Jian Pei ·

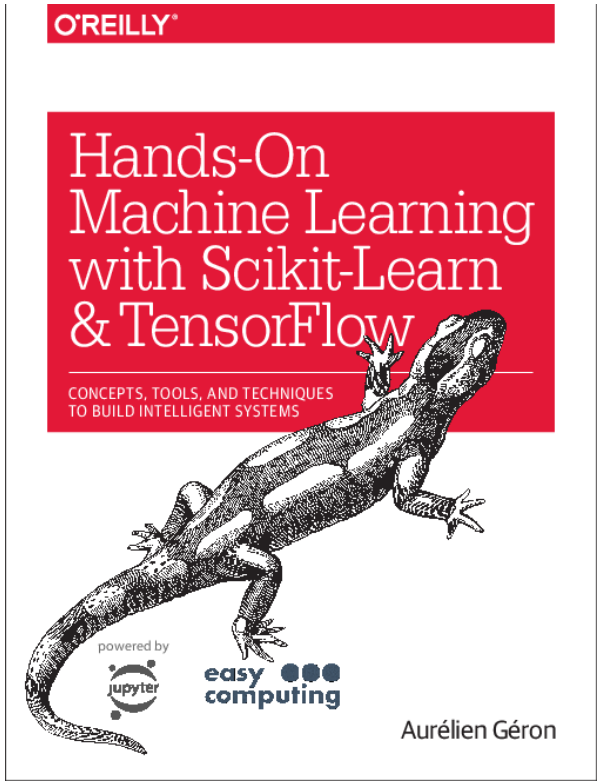


Introduction to Data Mining by Pang-Ning Tan Michael Steinbach Vipin Kumar

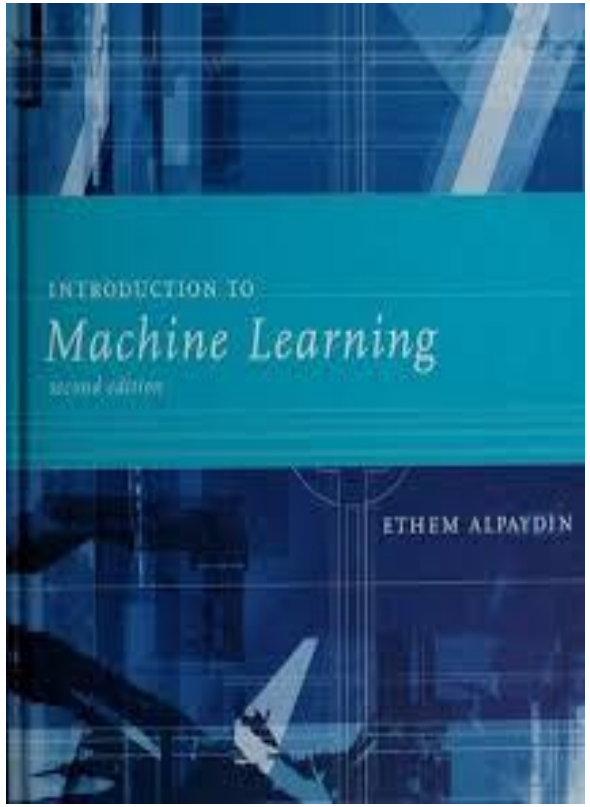
Course Materials and References



Introduction to Machine Learning with Python by Andreas C. Muller and Sarah Guido



Hands-On Machine Learning with Scikit-Learn and TensorFlow by Aurélien Géron



Introduction to Machine Learning by Ethem Alpaydın

Machine Learning

- What is Machine Learning
- Why Use Machine Learning
- Types of Machine Learning Systems
- Main Challenges of Machine Learning
- Machine Learning Cycle

- Classification
- Linear Models
- Underfitting vs Overfitting
- Model Evaluations
- K-Nnearest Neighbors Algorithm

What is Machine Learning

- Machine Learning is a subset of Data Mining focused on algorithms that learn patterns from data and generalize to unseen data.

- It gives computers the ability to learn without being explicitly programmed. (Arthur Samuel ,1959).

- Tom Mitchell (1998) Well-posed Learning Problem: A computer program is said to learn from experience E with respect to some task T and some performance measure P, if its performance on T, as measured by P, improves with experience E.

What is Machine Learning

- ML focuses specifically on predictive modeling.

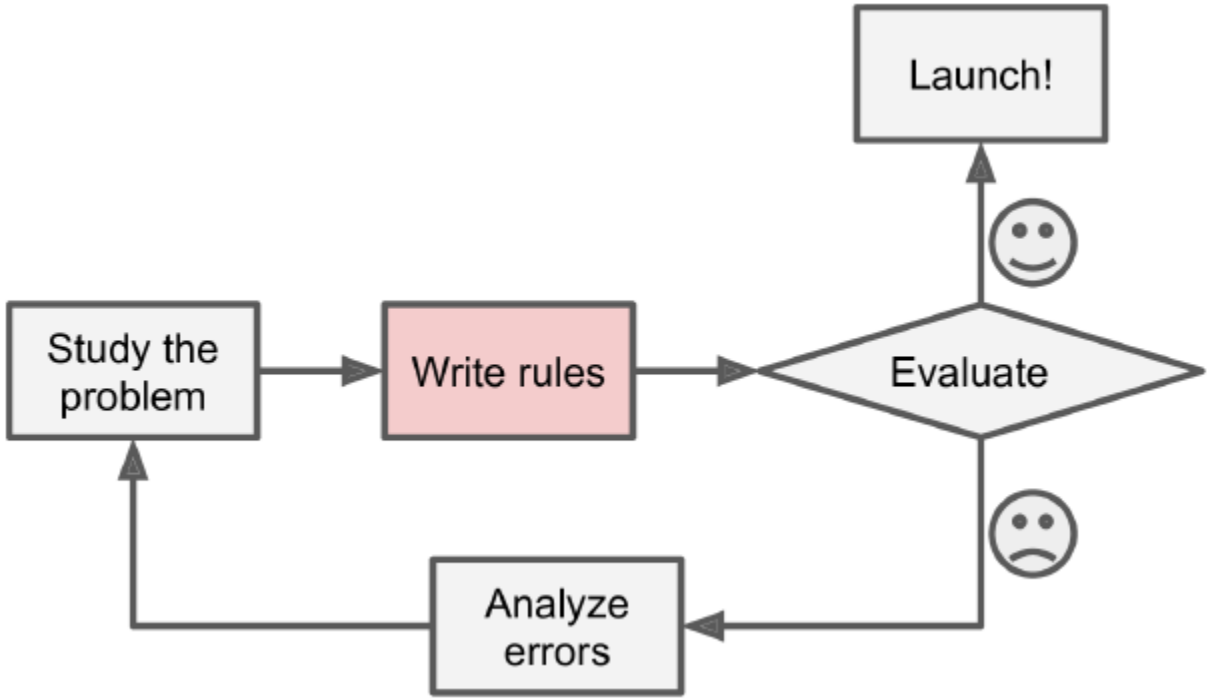
- Machine Learning:
  - Grew out of work in AI
  - New capability for computers
  - Data volume growth

Why Use Machine Learning?

- Take an example of spam filter

Traditional programming techniques:

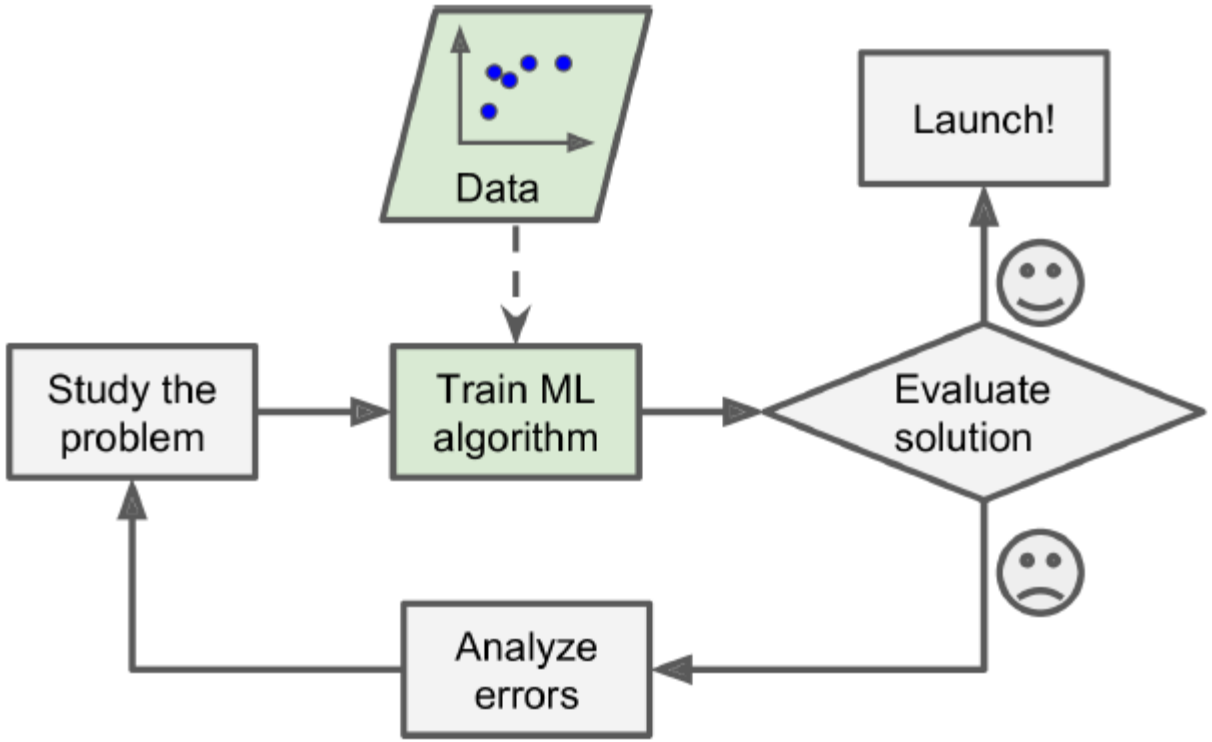
- Observe some words or phrases (“4U,” “credit card,” “free,” and “amazing”)
  – sender’s name, etc.
- Drawbacks:
  - Long list of complex rules
  - Hard to maintain



Why Use Machine Learning?

- ML Methods:
- Automatically learns which words and phrases are good predictors of spam
  - by detecting frequent patterns of words in the spam examples compared to the ham examples

- Advantages:
  - Shorter
  - Easier to maintain
  - More accurate



Machine Learning can be Used for:

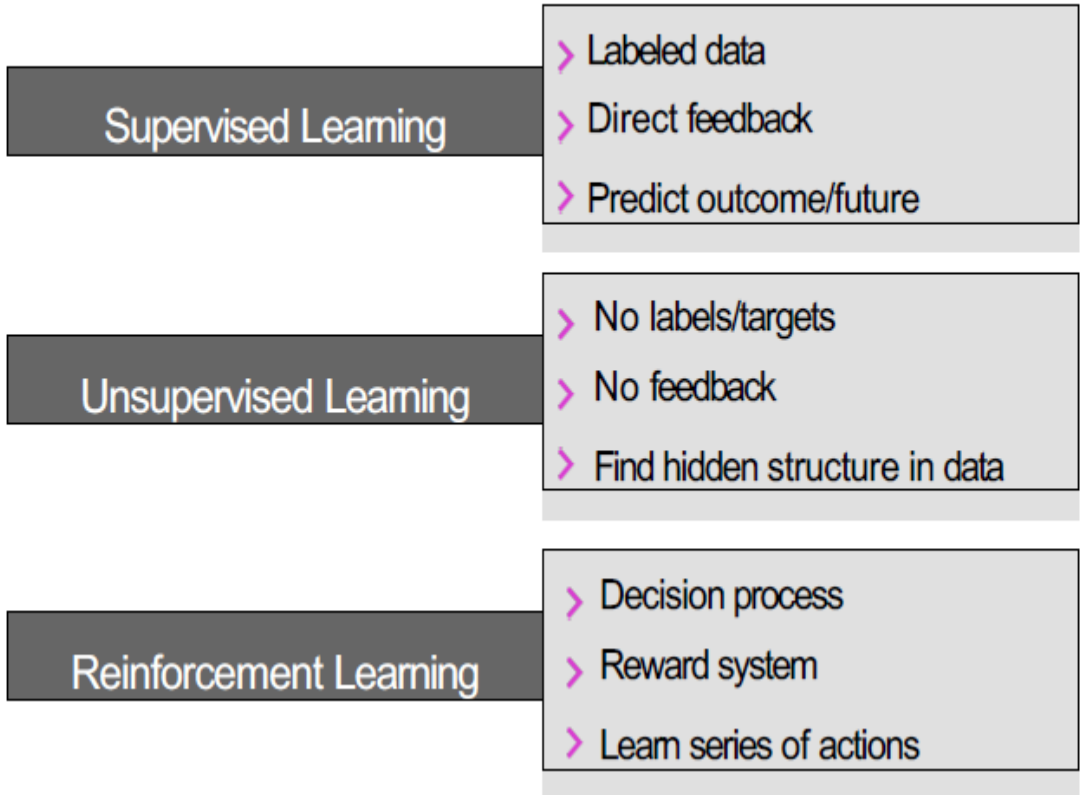
- Problems for which existing solutions require a lot of manual tuning or long lists of rules
  - Better performance
- Complex problems for which there is no good solution at all
- Fluctuating environments
  – Adaptive
- Getting insights about complex problems and large amounts of data
  – Discover unknown patterns

Types of Machine Learning Systems

- Trained with human supervision or not
  - supervised, unsupervised, semi-supervised, and Reinforcement Learning
- Can learn incrementally on the fly or not
  - Online (from a stream of incoming data) versus batch learning
- Working by comparing new data points to known data points, or detecting patterns in the training data and build a predictive model
  - instance-based versus model-based learning

Types of Machine Learning Tasks:

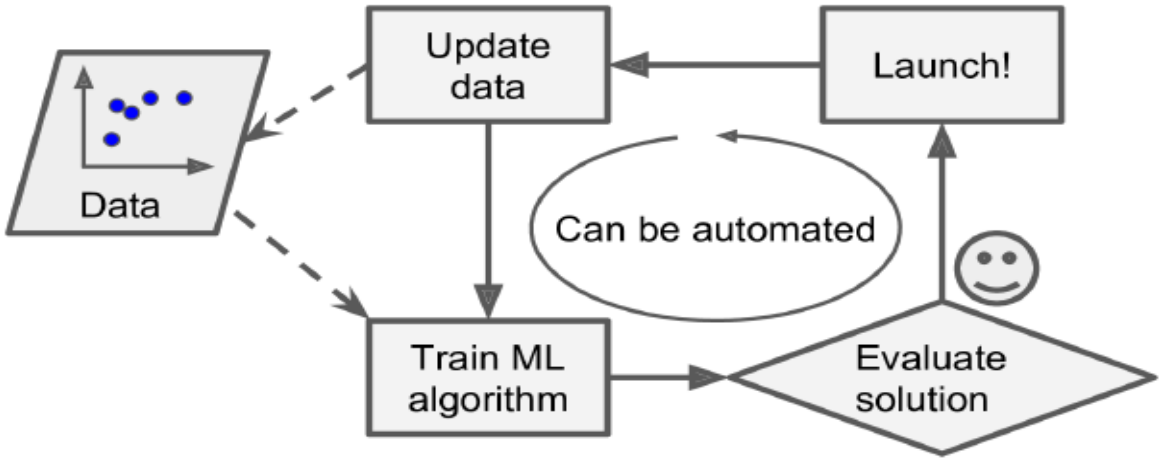
- ***Supervised Learning:***
  1. Regression (Continuous values)
  2. Classification (Categories)
- ***Unsupervised Learning:***
  1. Clustering
  2. Dimensionality reduction
  3. Anomaly detection
- Semi Supervised Learning
- Reinforcement Learning



Batch Learning

- It must be trained using all the available data (Offline learning).
- Properties
  - Usually takes a lot of time (many hours) and computing resources
  - Runs without learning anymore (after finishing training)
  - New data → Train a new version from the beginning on the whole dataset
  - The whole process can be automated
- Not suitable for:
  - Rapidly changing data
  - Huge amount of data
  - Limited resources

Solution: Online learning



Online Learning

- The system is trained incrementally by providing data sequentially
  - Raw by raw or in mini-batches
- Properties
  - Learning process is fast and cheap
  - Learning rate
    - High: rapidly adapt to new data
    - Low: less sensitive to noise

➢ Suitable for:

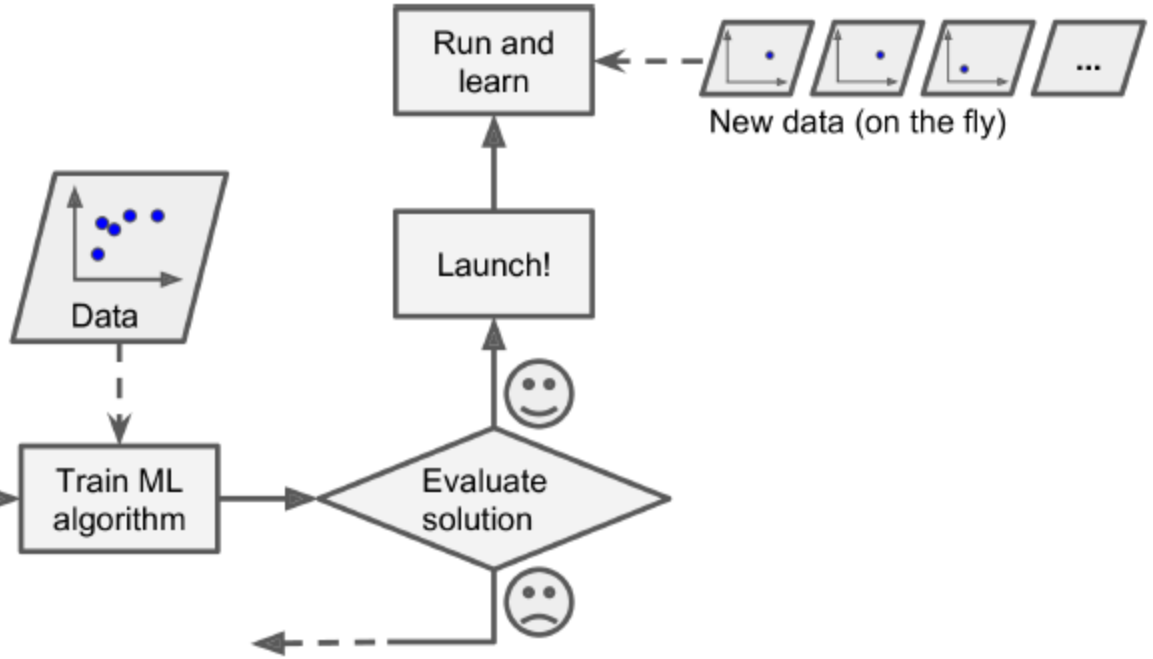
✓ Continuous data (e.g., stock prices)

✓ Rapidly changing data

✓ Autonomous learning

✓ Out-of-core learning (for huge data or limited resources)

  - Huge datasets that cannot fit machine's memory



Main Challenges of Machine Learning

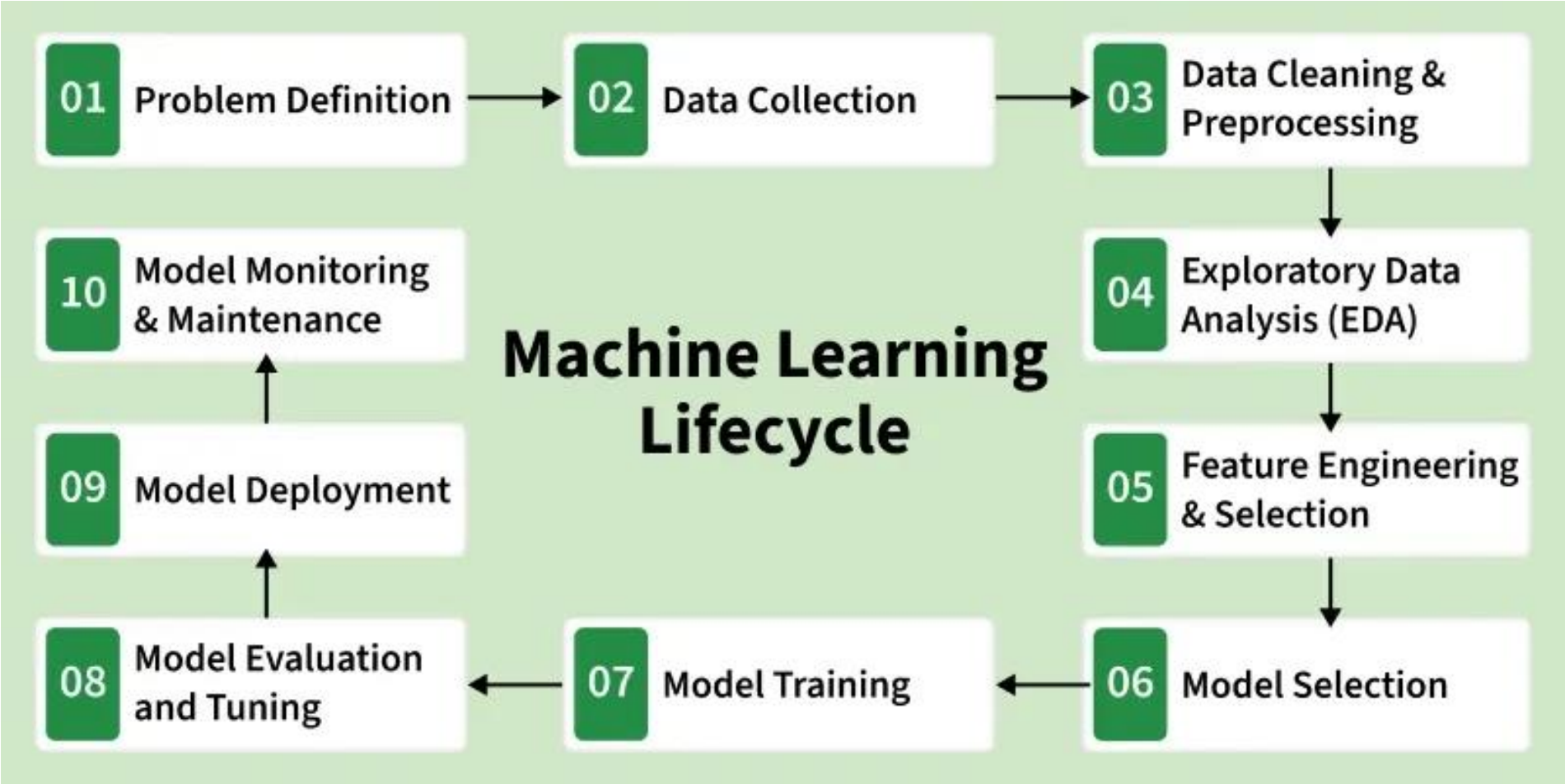
- Data Issues

1) Insufficient Quantity of Training Data
2) Non-representative Training Data
3) Poor-Quality Data
4) Irrelevant Features
5) Data Mismatch

- Algorithms

1) Overfitting the Training Data
2) Underfitting the Training Data

Machine Learning Cycle



Prediction Types : Classification vs. Numeric Prediction

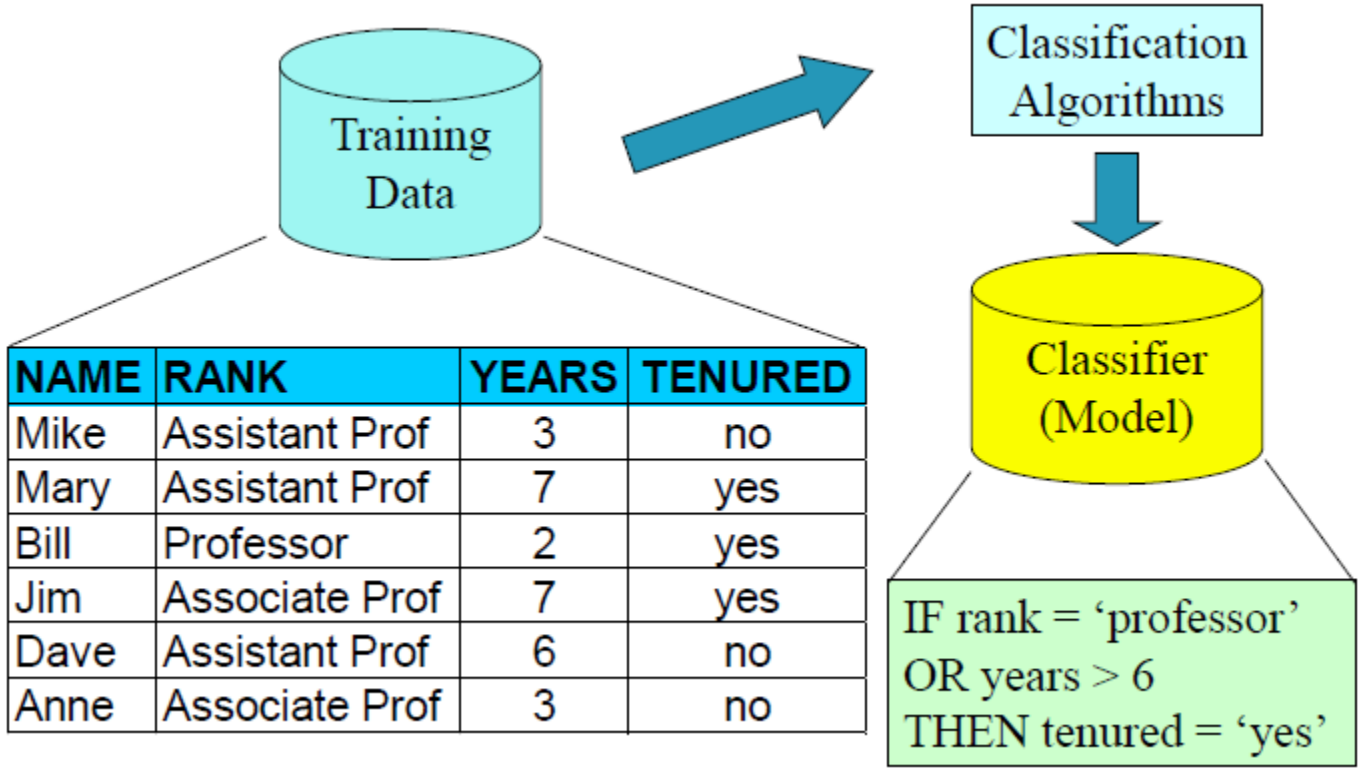
- Classification
  - Predicts categorical class labels (discrete or nominal)
  - Classifies data based on the training set (constructs a model) and the values (class labels) in a classifying attribute and uses it in classifying new data
- Numeric Prediction
  - Models continuous valued functions, i.e., predicts unknown or missing values

Classification—A Two-Step Process

1) Model construction : describing a set of pre determined classes
   - Each tuple/sample is assumed to belong to a predefined class, as determined by the class label attribute
   - The set of tuples used for model construction is training set
   - The model is represented as classification rules, decision trees, or mathematical formulae
2) Model usage : for classifying future or unknown objects
   - Estimate accuracy of the model
     - The known label of test sample is compared with the classified result from the model
     - Accuracy rate is the percentage of test set samples that are correctly classified by the model
     - Test set is independent of training set (otherwise overfitting)
   - If the accuracy is acceptable, use the model to classify new data

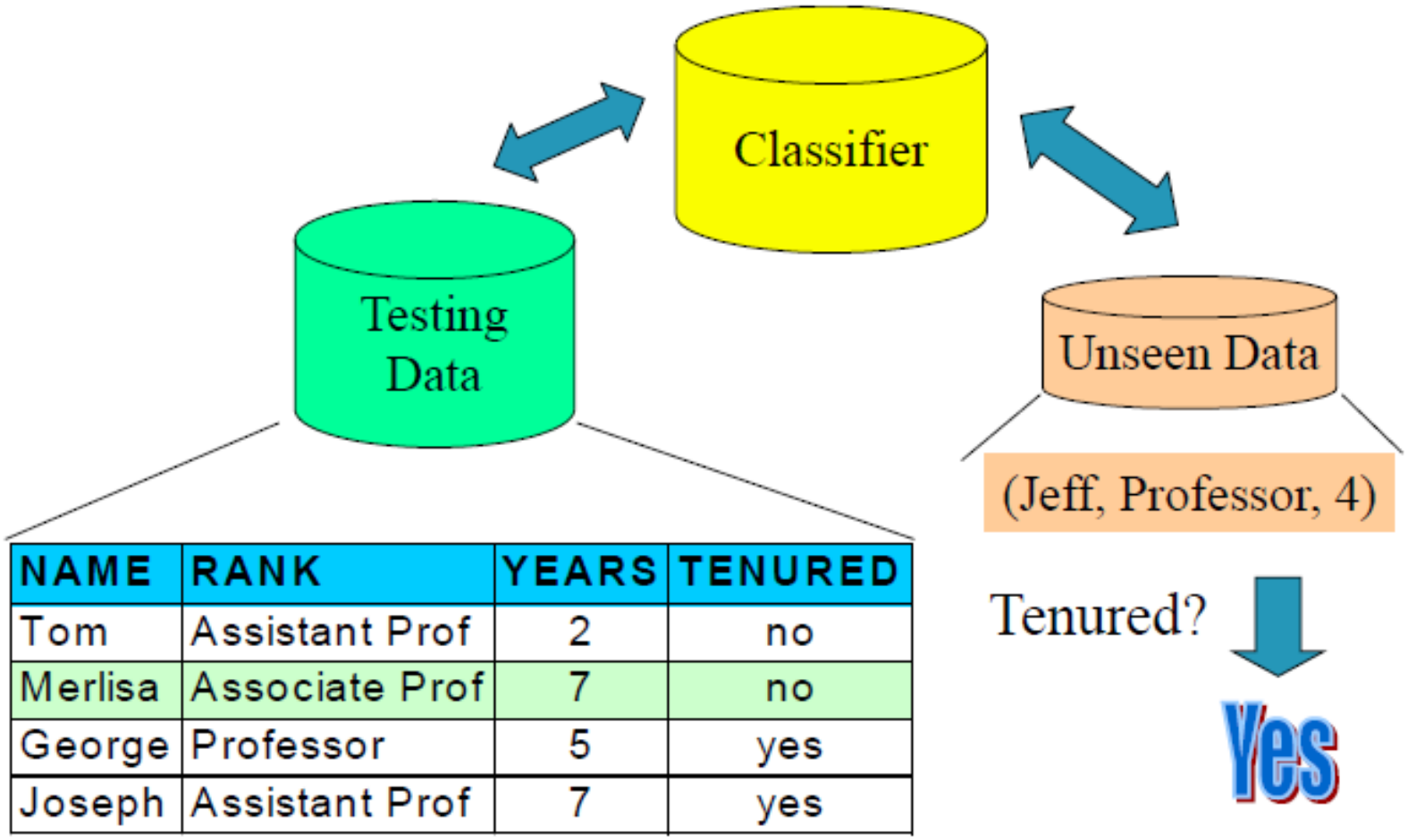
- Note: If the test set is used to select models, it is called validation set

Process (1): Model Construction



NAME	RANK	YEARS	TENURED
Mike	Assistant Prof	3	no
Mary	Assistant Prof	7	yes
Bill	Professor	2	yes
Jim	Associate Prof	7	yes
Dave	Assistant Prof	6	no
Anne	Associate Prof	3	no

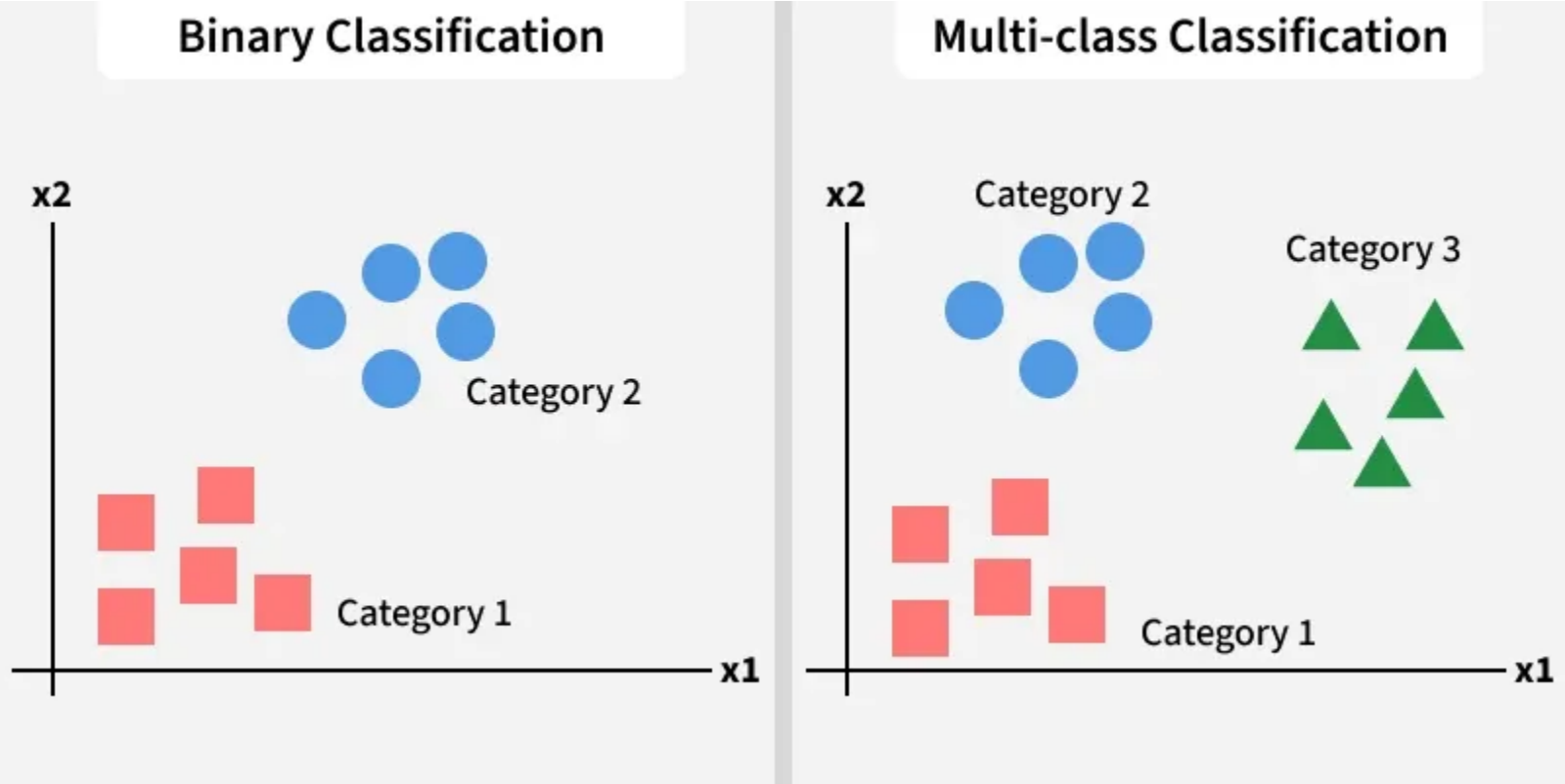
Process (2): Using the Model in Prediction



NAME	RANK	YEARS	TENURED
Tom	Assistant Prof	2	no
Merlisa	Associate Prof	7	no
George	Professor	5	yes
Joseph	Assistant Prof	7	yes

Types of Classification

1. Binary Classification (Two options)
2. Multiclass Classification (More Than Two)



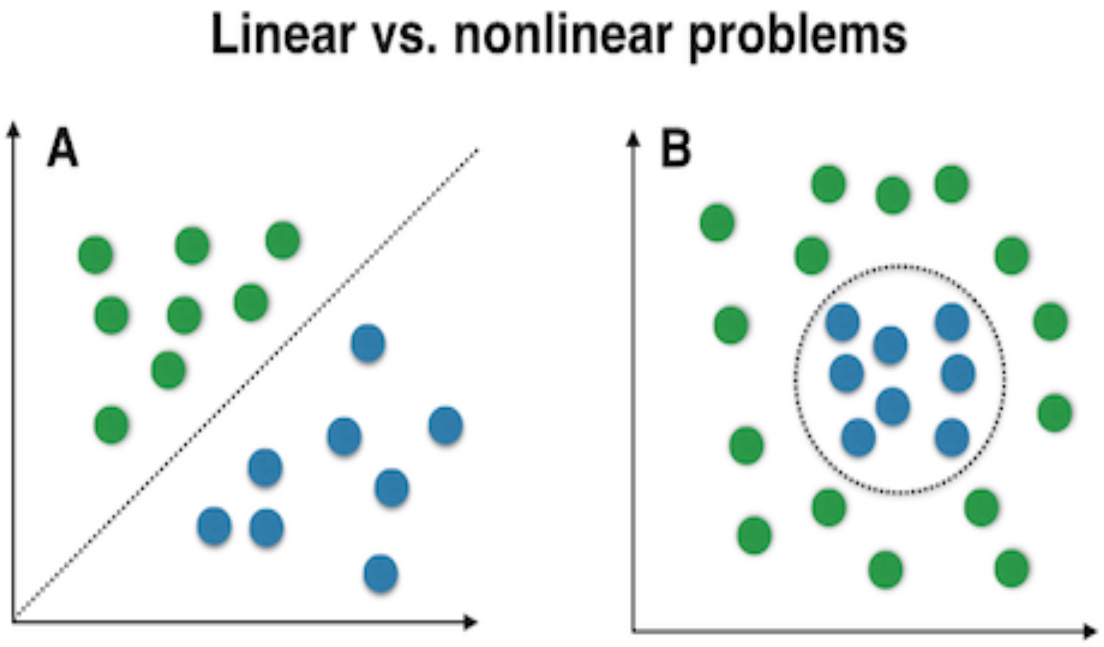
Binary Classification

Can a Line Separate Classes?

y=f(x), where y is the categorical output

Types of ML classification algorithms :

1. Linear models:
   - Logistic Regression
   - Support Vector Machine
2. Non-Linear Models
   - K-Nearest Neighbors
   - Decision Tree
   - Random Forest
   - Naive Bias



Linear Models

We have data like:

Hours Studied	Passed Exam
2	0
5	1
3	0
8	1

We want a rule that says:
Given hours studied → predict 0 or 1.

Linear Models

$$z=\omega*x+b$$

Prediction score = (weight × input) + bias

If :

- ω is positive → more hours increases score
- ω is negative → more hours decreases score

Linear Models

- Example:

If $w = 1$ and $b = -4$ Then:

- $x = 2 \rightarrow z = -2$ So low hours → negative score
- $x = 8 \rightarrow z = 4$ High hours → positive score

Already this separates classes.

If $z > 0 \rightarrow$ predict 1
If $z < 0 \rightarrow$ predict 0

That's a classifier.

The boundary is where: $z = 0$

Which means: $\omega x + b = 0$

That's just the dividing point.

Probability

Problem:

The model says:

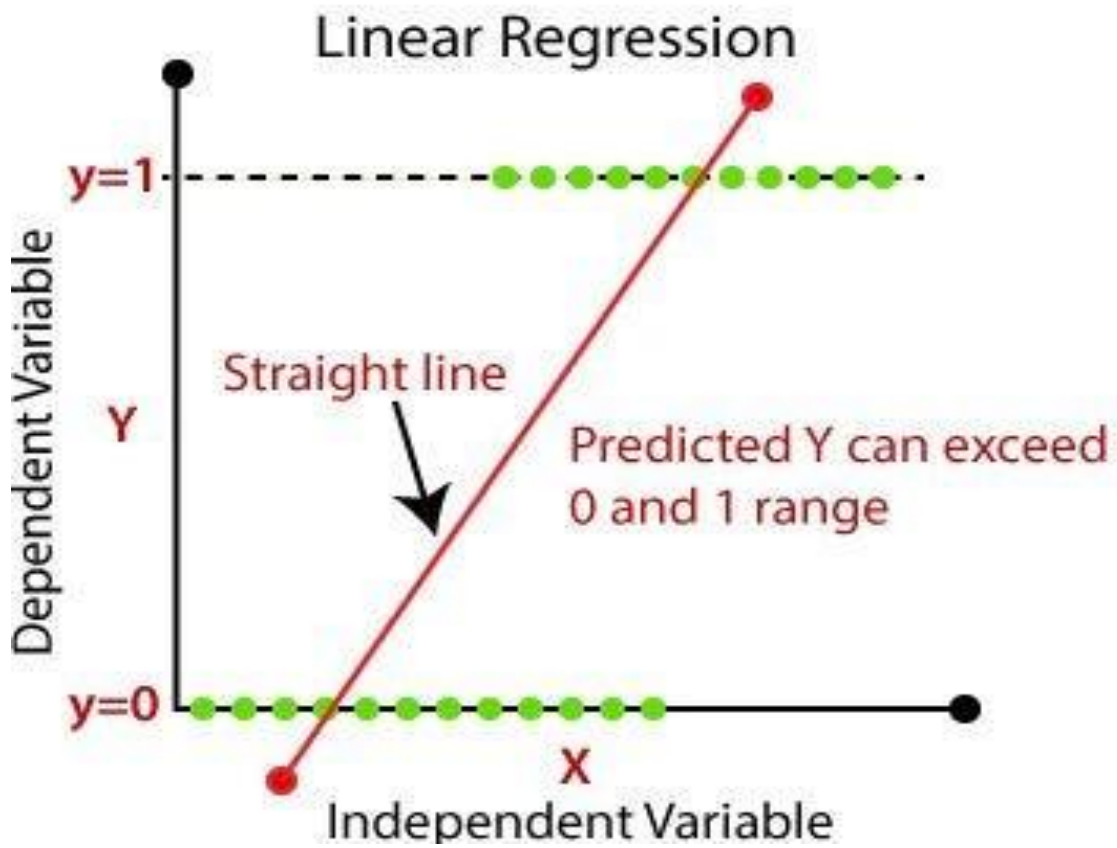
$x = 4.1 \rightarrow$ class 1

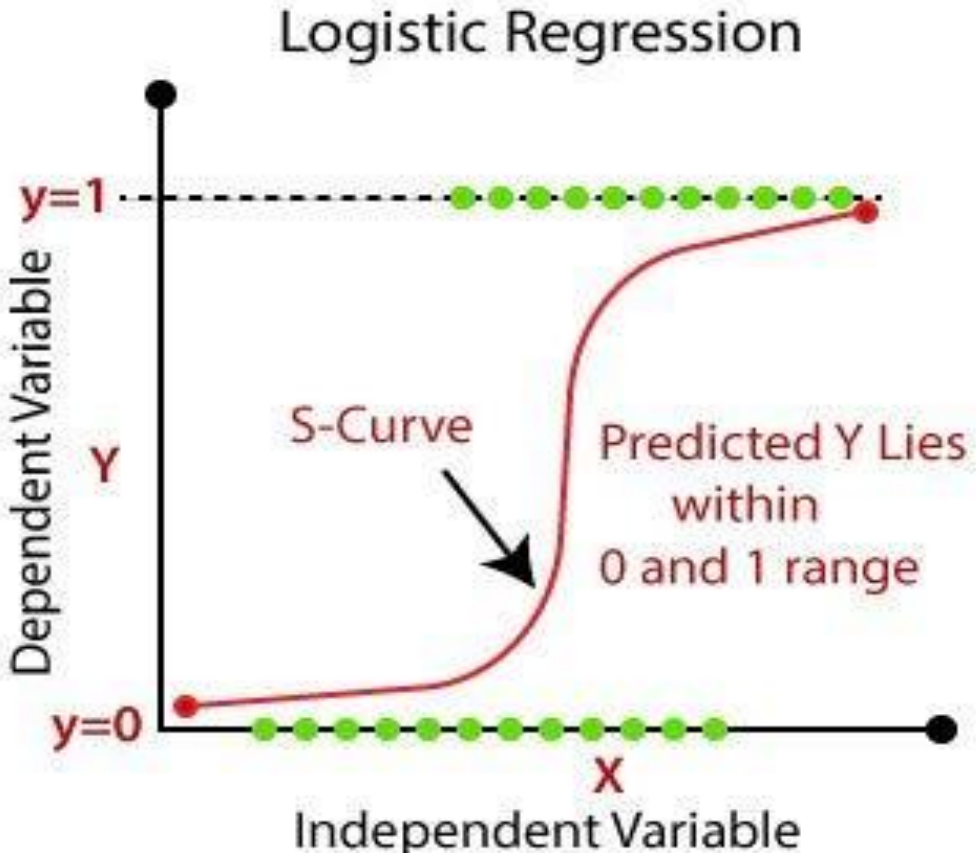
$x = 3.9 \rightarrow$ class 0 But they are almost identical.

Instead of hard 0 or 1, we want: A number between 0 and 1.

- 0.9 → very confident pass
- 0.6 → maybe pass
- 0.2 → likely fail

Logistic Regression





Interpretation: maps (-∞, ∞) to (0,1)

Logistic Regression

$$\sigma(z) = \frac{1}{1+e^{-z}} \text{ , } z = \omega * x + b$$

How Do We Make It Learn? (How do we choose ω and b?)

We introduce penalty when prediction is wrong.

- Case 1:

True label = 1

- Model says probability = 0.9 → good
- Model says probability = 0.1 → very bad

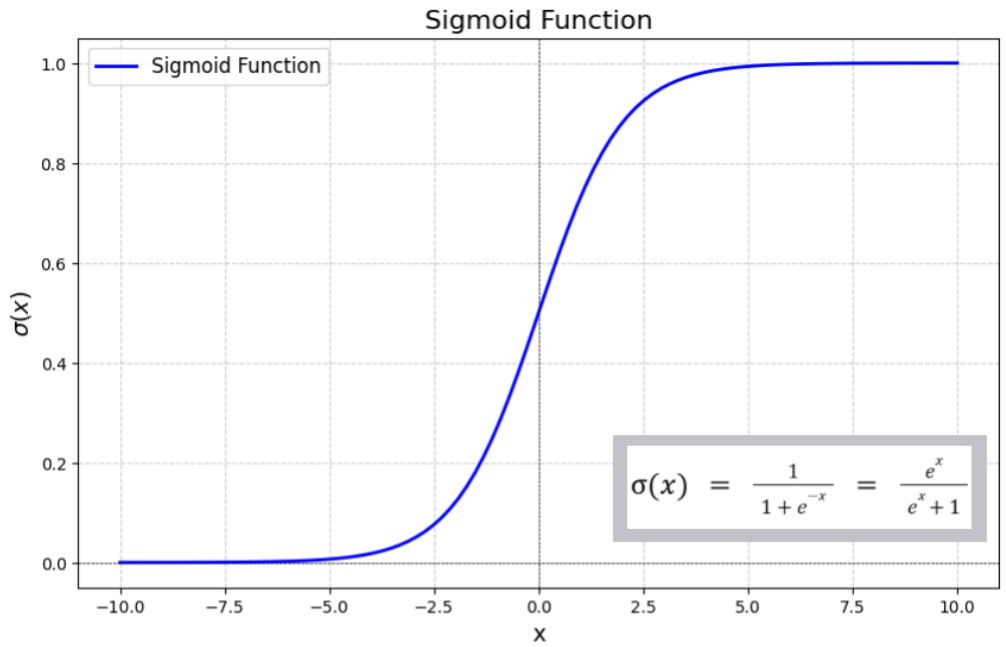
- Case 2:

True label = 0

- Model says probability = 0.1 → good
- Model says probability = 0.9 → very bad

We want a function that:

Punishes confident wrong predictions strongly.



Cross-Entropy

That function is:

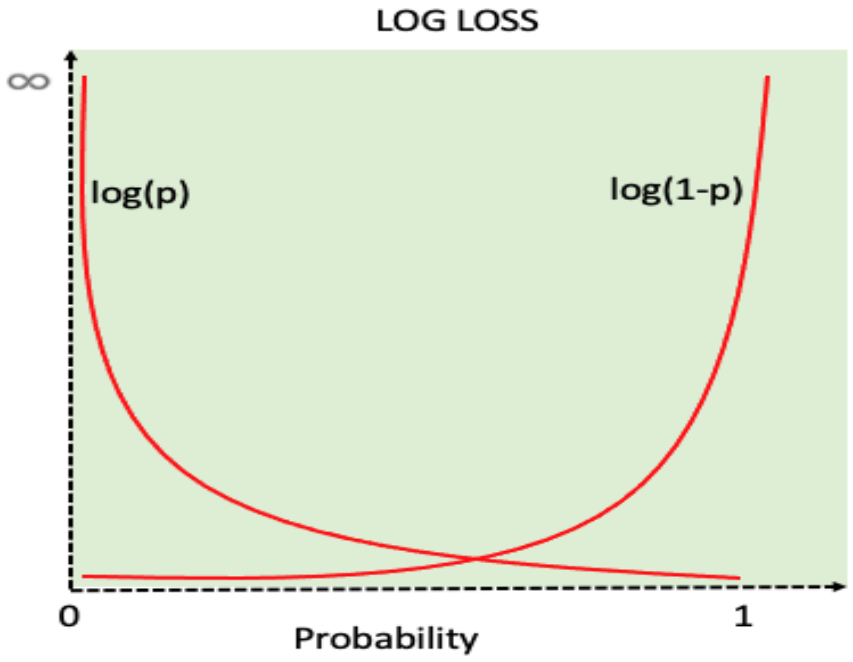
$$H(P,Q) = -\sum_i y_i * log(p_i)$$

$$Loss = -(y * \log(p) + (1-y) * \log(1-p))$$

- Where

y = true label (0 or 1)

p = predicted probability



We want a loss that:

- is small when prediction is correct
- becomes very large when we are confidently wrong

Example:

Two classes [Cat, Dog] : Dog is correct

$$Q = [0,1]$$

Prediction:

$$P = [0.9,0.1]$$

Cross entropy becomes:

$$H = -(0*\log(0.9)+1*\log(0.1))$$

$$H = -\log(0.1) = 2.302$$

If prediction was correct:

$$P = [0.1,0.9]$$

$$H = -\log(0.9) = 0.105$$ Much smaller.

Gradient Descent

- Our loss for one data point is:

$$L = -[\mathrm{y} * \log(p) + (1 - y) * \log(1 - p)] =$$

Where:

$$p = \sigma(z)$$

$$z = \omega * \mathrm{x} + \mathrm{b}$$

- So there is a chain:

$$\omega \rightarrow z \rightarrow \mathrm{p} \rightarrow \mathrm{L}$$

If we change ω a little bit, how does the loss change?

Gradient Descent

- Mathematically:

$$\frac{dL}{d\omega}$$

- So we follow the chain.

Because w affects z.
z affects p.
p affects L.

- So we use the chain rule:

$$\frac{dL}{d\omega} = \frac{dL}{dp} \cdot \frac{dp}{dz} \cdot \frac{dz}{d\omega}$$

Gradient Descent

$$L = -[\mathrm{y} * \log(p) + (1 - y) * \log(1 - p)]$$

Derivative with respect to p:

$$\frac{dL}{dp} = -(\frac{y}{p} - \frac{1-y}{1-p})$$

Derivative with respect to z:

$$\mathrm{p}=\sigma(\mathrm{z})=\frac{1}{1+\mathrm{e}^{-\mathrm{z}}}$$

$$\frac{dp}{dz}=\mathrm{p}*(1\text{-}\mathrm{p})$$

Derivative with respect to ω:

$$\frac{dz}{d\omega} = x$$

Multiply Everything

$$\frac{dL}{d\omega} = -\left(\frac{y}{p} - \frac{1-y}{1-p}\right).\mathrm{p}.(1-\mathrm{p}).\mathrm{x}$$

After simplifying

$$\frac{dL}{d\omega} = (p-y).x$$

- Thus the gradient is:

(Prediction − Truth) times input

- If:

  Prediction too large → gradient positive → weight decreases
  Prediction too small → gradient negative → weight increases

- So learning becomes:

$$w := w - \eta(p-y)x := w - \eta\frac{dL}{d\omega}$$

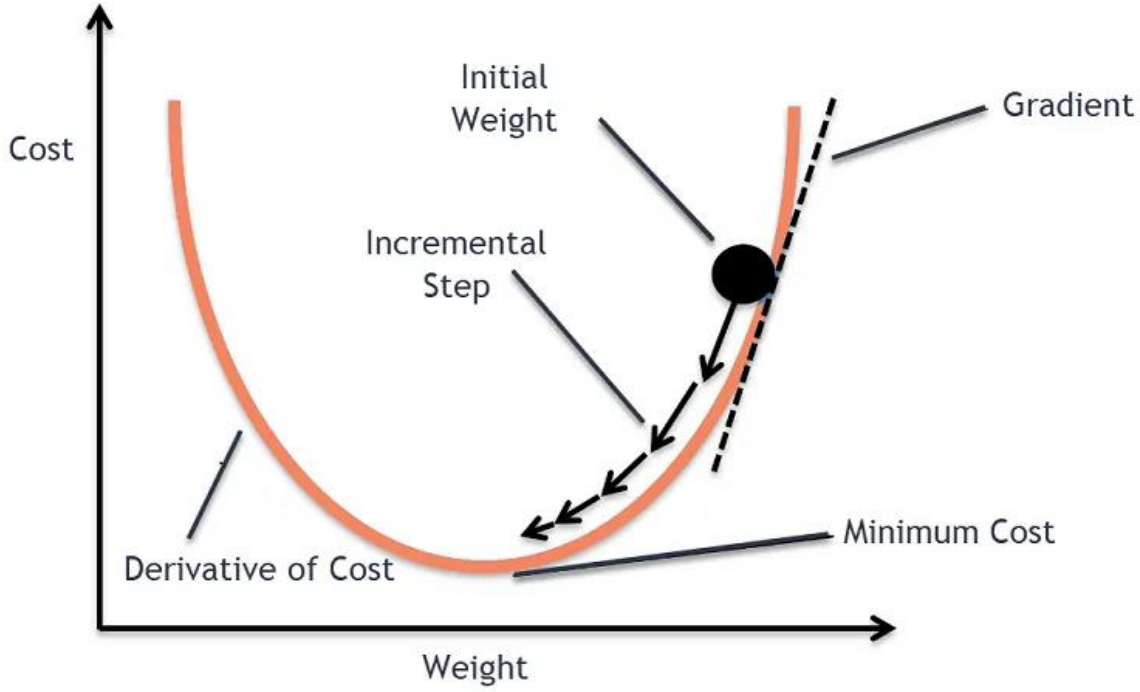
η: Learning Rate

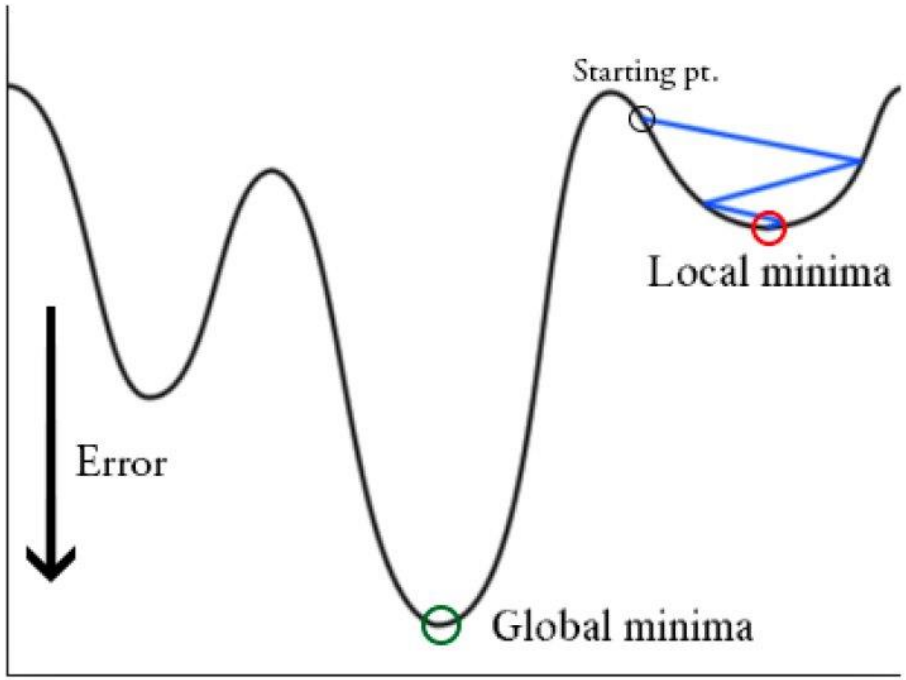
$$New\ weight = Old\ weight - \text{Learning Rate} * [(prediction - truth) * \text{input}]$$

Same for bias b

$$b := b - \eta\frac{dL}{db}$$

Gradient Descent





How Does This Work for the Whole Dataset?

We have N samples then Loss over dataset:

$$J = \frac{1}{N}\sum_{i=1}^{N} L_i$$

Gradient becomes:

$$\frac{dJ}{dw} = \frac{1}{N}\sum_{i=1}^{N}(p_i - y_i)x_i$$

Bias:

$$\frac{dJ}{db} = \frac{1}{N}\sum_{i=1}^{N}(p_i - y_i)$$

Multiple Features

Instead of one feature x, we have:

$$x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix}$$

Weights:

$$\omega = \begin{bmatrix} \omega_1 \\ \omega_2 \\ \vdots \\ \omega_d \end{bmatrix}$$

Linear model:

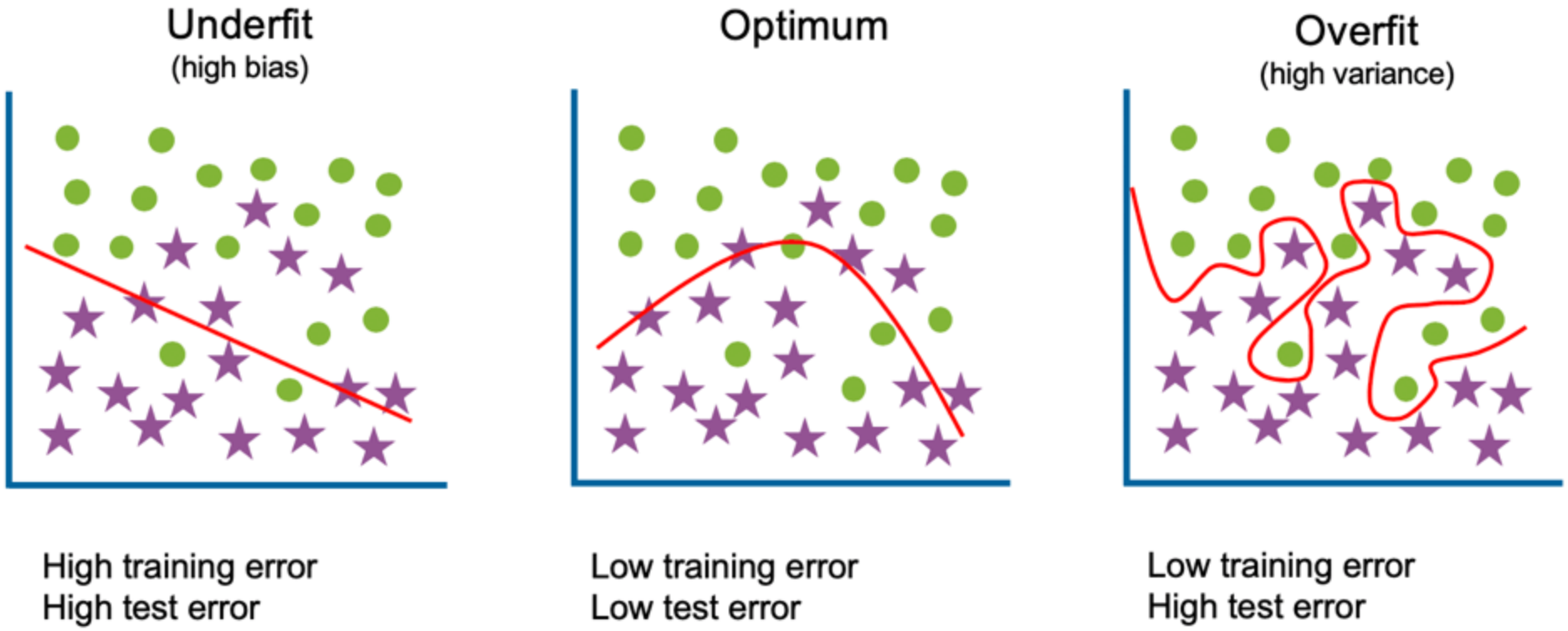
$$z = \omega^t x + b$$

Which expands to:

$$\omega_1 x_1 + \omega_2 x_2 + \cdots + \omega_d x_d + b$$

Underfitting vs Overfitting

- If our model performs perfectly on training data, is that enough?



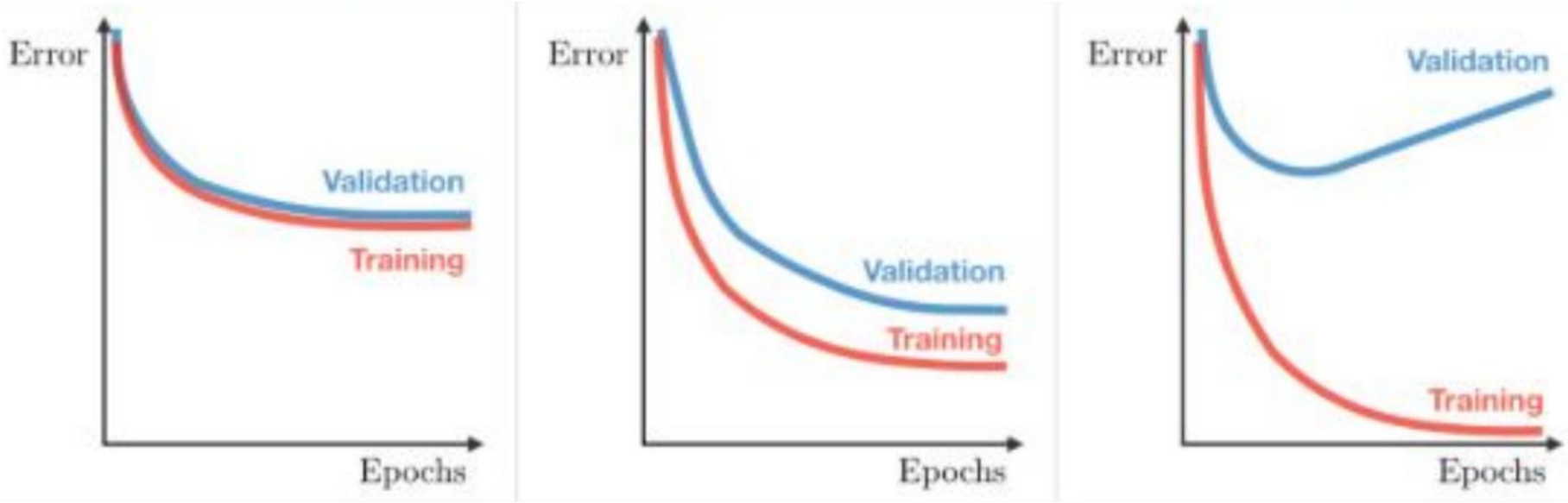
Straight line trying to fit curved data.

Crazy wiggly boundary fitting every point.
Small dataset

Underfitting vs Overfitting

- Underfitting
  - The model is too simple to capture the true pattern.
  - Not enough training time
  - Too few features

- Overfitting
  - The model memorizes the training data instead of learning the pattern.
    - Poor performance on new (test) data
    - Model is too complex
    - Too many features
    - Not enough training data



Bias and Variance

Bias

Error due to model assumptions.

In linear models:

Assuming data is linearly separable when it is not.

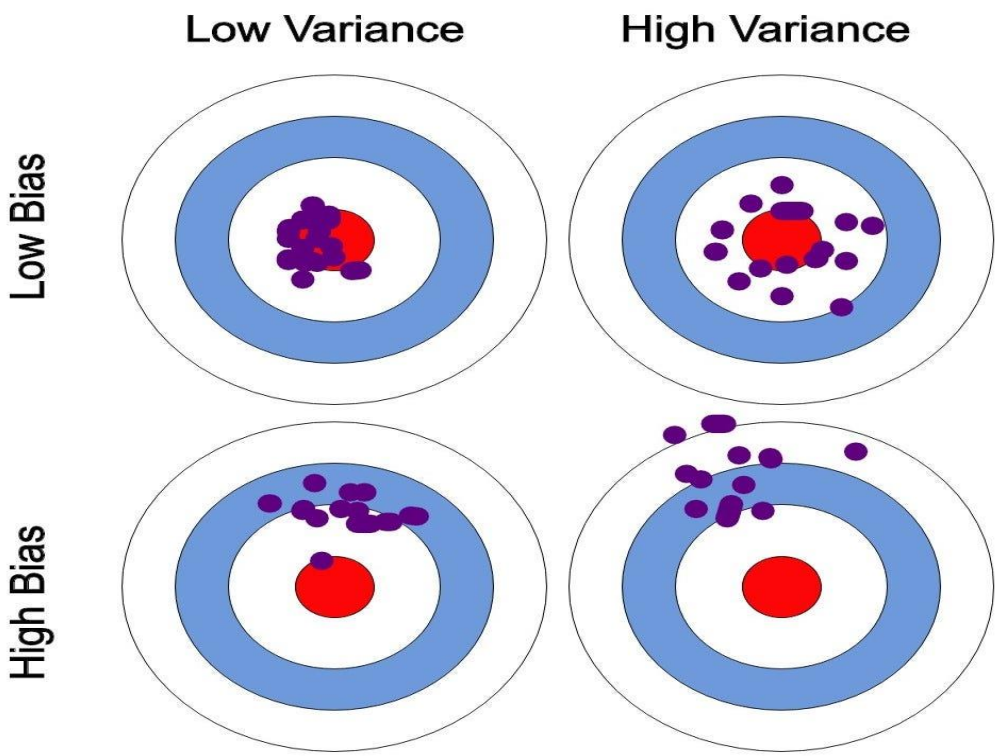
High bias → underfitting.

Variance

Error due to sensitivity to data fluctuations.

If dataset is changed slightly and model changes drastically → high variance.

High variance → overfitting



Addressing overfitting

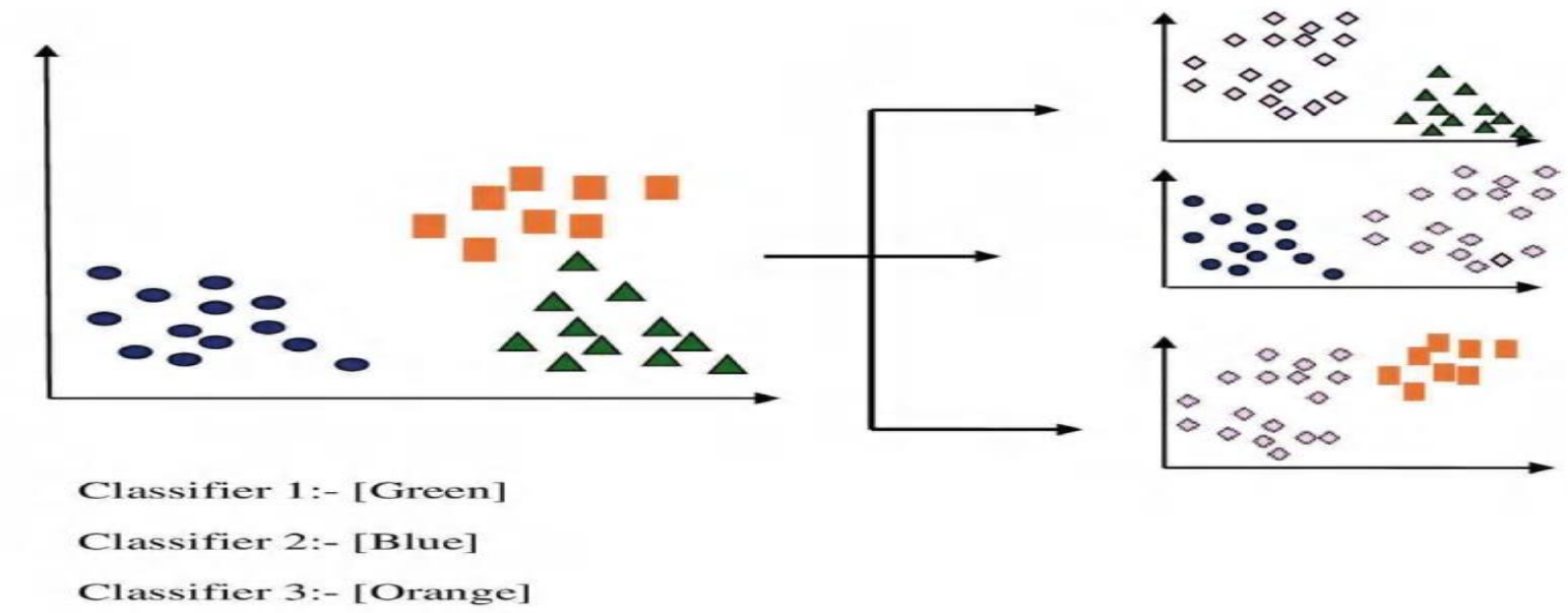
1. Collect more data.
2. Reduce number of features.
   — Manually select which features to keep.
   — Model selection algorithm
3. Regularization.
   — Keep all the features but reduce magnitude/values of parameters .
   — Works well when we have a lot of features, each of which contributes a bit to predicting

$$J = CrossEntropy + \lambda \mid\mid w \mid\mid^2$$

$$\lambda \mid\mid w \mid\mid^2 = \lambda \sum_j \omega_j{}^2$$

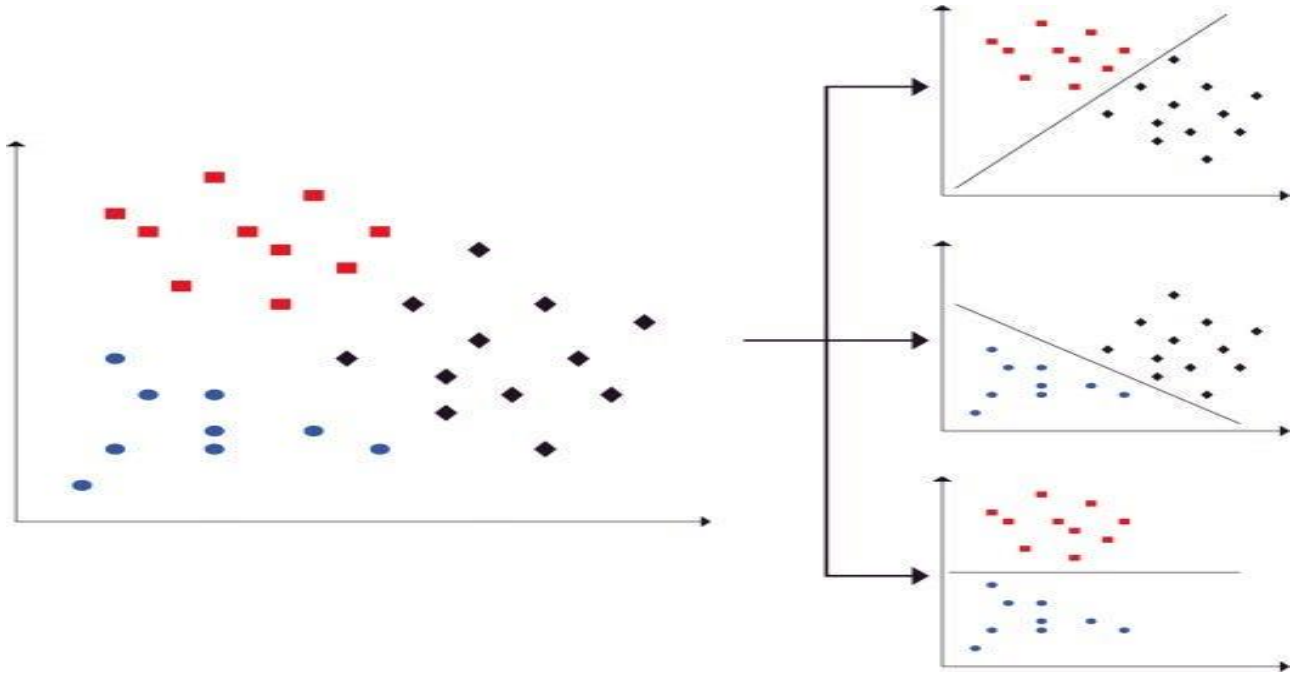
Binary Classifiers for Multi-Class Classification

- Binary classification models do not support multi-class classification
- One-vs-Rest strategy
  - Splits a multi-class classification into one binary classification problem per class.
  - One model to be created for each class.



Binary Classifiers for Multi-Class Classification

- One-vs-One strategy
  - Splits a multi-class classification into one binary classification problem per each pair of classes.
  - Splits the dataset into one dataset for each class versus every other class.
  - More datasets
  - **- N-class** instances dataset, $\frac{\boldsymbol{N*(N-1)}}{\mathbf{2}}$ binary classifier models.



K-Nearest Neighbors

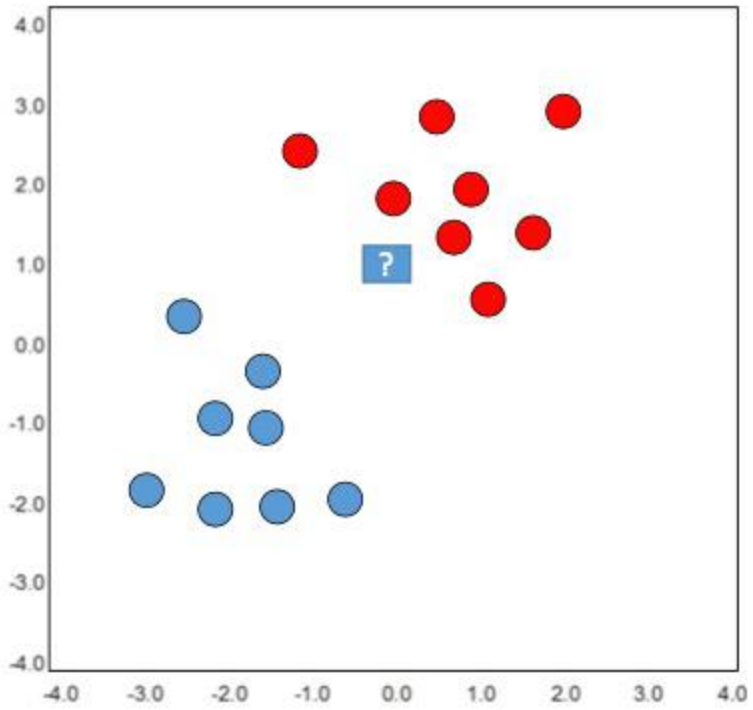
- K-Nearest Neighbors (KNN)
- How it Works?
- Distance Metrics
- K-NN Computational complexity

K-Nearest Neighbors (KNN)

- Instance-based learning:
  – Store training examples and delay the processing (“lazy evaluation”) until a new instance must be classified
- Two classes, two features

We want to assign label to unknown data point?

- Label should be red.



K-Nearest Neighbors (KNN)

Idea:

- We have similar labels for similar features.
- We classify new test point using similar training data points.

Algorithm overview:

- Given some new test point **x** for which we need to predict the class **y**.
- Find **most similar** data-points in the training data.
- Classify x "**like**" these most similar data points.

Questions:

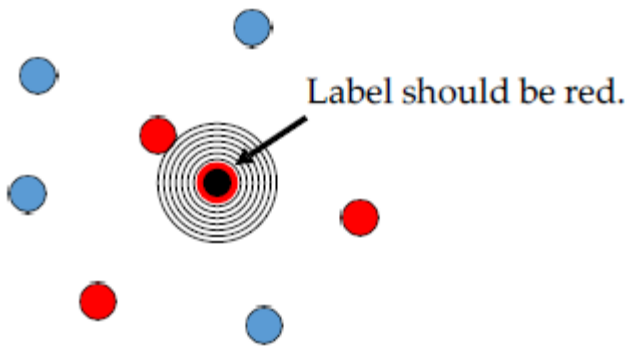
- How do we determine the similarity?
- How many similar training data points to consider?
- How to resolve inconsistencies among the training data points?

K-Nearest Neighbors (KNN)

❖ 1-Nearest Neighbor:

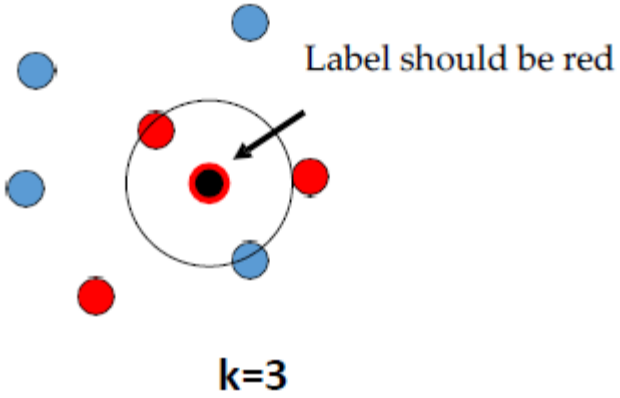
Simplest ML Classifier

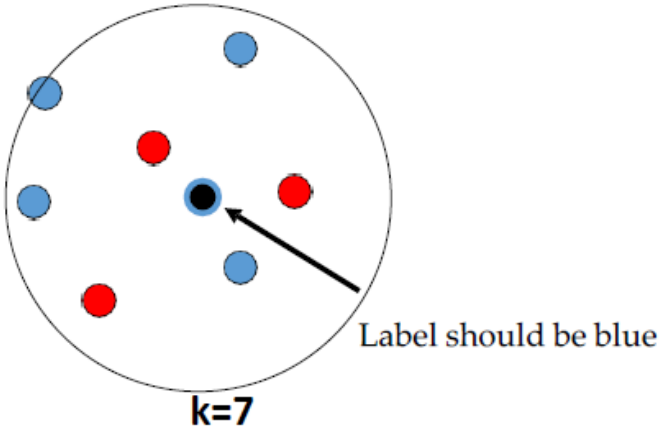
Idea: Use the label of the closest known point.



❖ Generalization:

Determine the label of k nearest neighbors and assign the most frequent label





Formal Definition:

- Using the S_x , we can define a classifier as a function that gives us the most frequent label of the data points in S_x

$$h(x) = mode(\{y' : (x', y') \in S_x\})$$

- $h(x)$ means the predicted label (output) for the input x.
- S_x represents the set of the K nearest training points to x.
- Each element in the dataset is written as: (x', y')
  - x' = the features of a training example
  - y' = the label of that training example
- $\{y' : (x', y') \in S_x\}$ ⇒ "Take the labels y'y' of all points whose pairs (x', y') are in the neighbor set Sx."

How it Works?

A. Selecting appropriate K which is number of nearest neighbors
   - Higher values of K reduces the noisy data and make the predictions more stable.
   - If k is too large the model may become too simple and miss important patterns (Underfitting)

Use for selecting K:

1. Cross-Validation
2. Elbow Method
3. Odd Values for k

Distance Metrics

B. Calculating the distance of new data point,

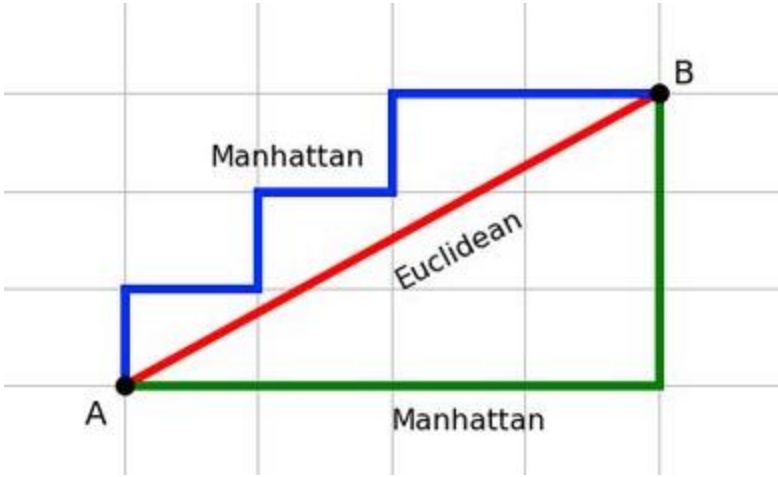
To identify nearest neighbor we use below distance metrics:

1. Euclidean Distance

$$d(x, y) = \sqrt{\sum_{j=1}^{d} (x_j - y_j)^2}$$

2. Manhattan Distance

$$d(x, y) = \sum_{i=1}^{n} | x_i - y_i |$$



Distance Metrics

3. **Minkowski Distance**

$$\mathrm{d}(\mathrm{x},\mathrm{y}) = \left(\sum_{i=1}^{n} |x_i - y_i|^p\right)^{\frac{1}{p}}$$

Where:

x_i, y_i = feature values

n = number of features (dimensions)

p = parameter that controls the type of distance

p value	Distance Type
p=1	Manhattan Distance
p=2	Euclidean Distance
p→∞	Chebyshev Distance

Distance Metrics

4. Cosine Distance

$$\text{cosine distance} = 1 - \text{cosine similarity}$$

$$cosine\ similarity(x, y) = \frac{x \cdot y}{||x||\ ||y||}$$

Where:

- x·y = dot product
- $||x||$,||y|| = vector magnitudes

Values range from:

1 → vectors point in the same direction

0 → vectors are orthogonal (unrelated)

-1 → opposite direction

Distance Metrics

Properties of Distance Metrics

- Nonnegative , dist(x,x') ≥ 0
- Symmetric , dist(x,x') = dist(x',x)
- dist(x,x') =0 $\Leftrightarrow x = x'$

- **Practical issues in computing distance:**
  - **Mismatch in the values of data**
    - **Issue:** Distance metric is mapping from d-dimensional space to a scaler. The values should be of the same order along each dimension.
  - **Solution:** Data Normalization

How it Works?

C. Select the K nearest neighbors based on distance calculated

D. Voting for Classification or Taking Average for Regression

Advantages:

- Can be used for both classification and regression
- Non parametric
- Works well on multi classes algorithms.
- unbiased in nature and also doesn’t make prior assumptions

- Disadvantages
  - Measuring the distance between all data points in training samples is a computationally expensive task
  - Requires more memory

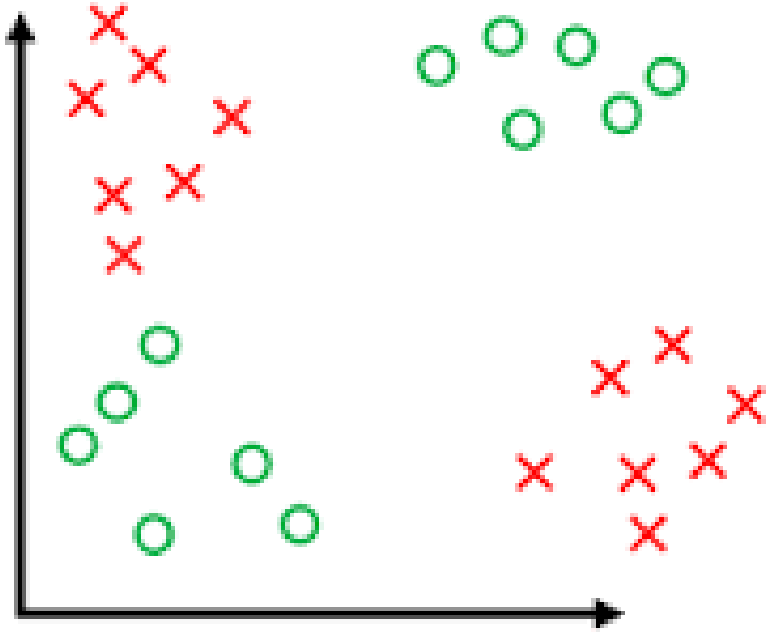
K-NN Computational complexity:

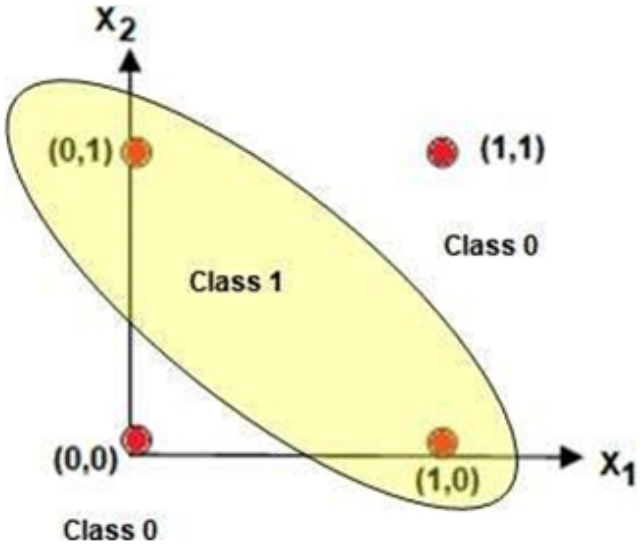
- Training Complexity
  - Consists of **storing the dataset** → O(1)
- **Prediction**
  - Compute distance from the query point to all n samples → O(nd)
  - Find the k smallest distances → about O(n)
- **Total prediction complexity** →O(nd)
- **Memory**

The algorithm must store the dataset: → O(nd)

What if the data is NOT linearly separable?

Logistic regression gives one straight boundary.





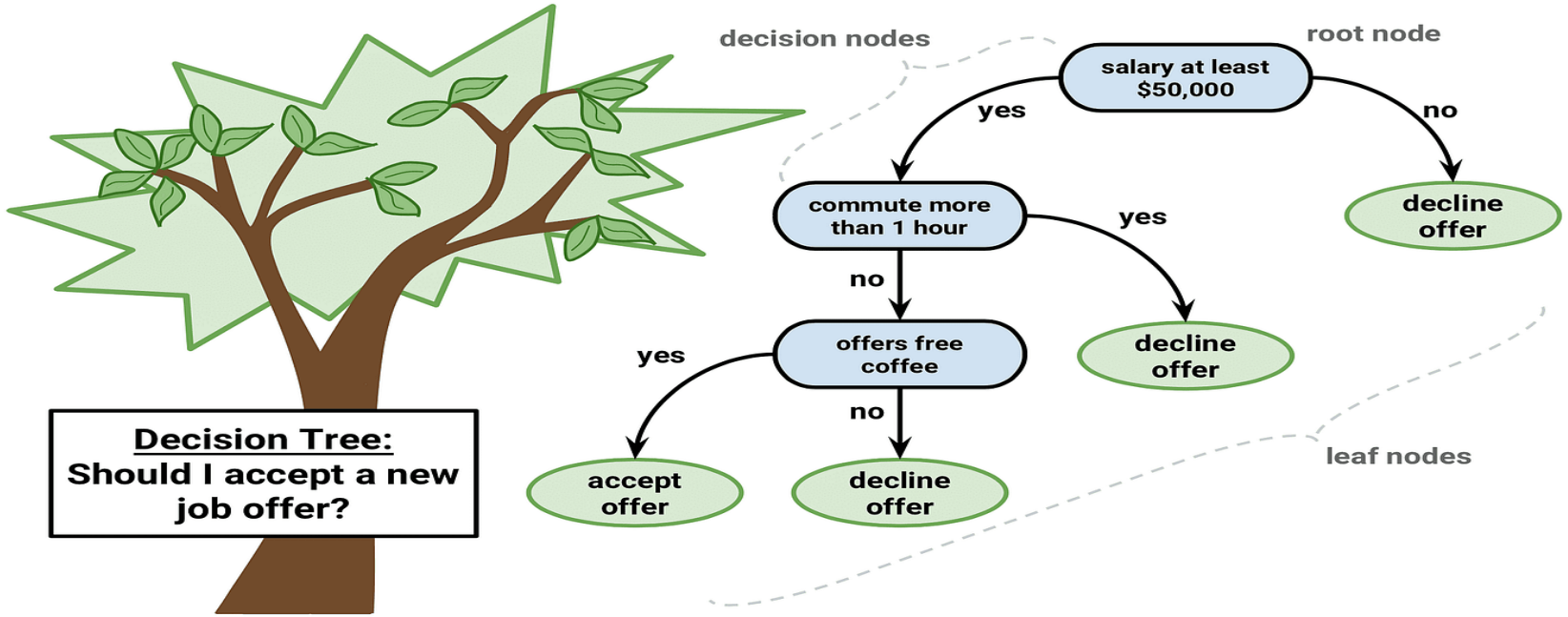
Truth Table for XOR Gate		
X1	X2	Output Class
0	0	0
0	1	1
1	0	1
1	1	0

What if we want boundaries that bend?

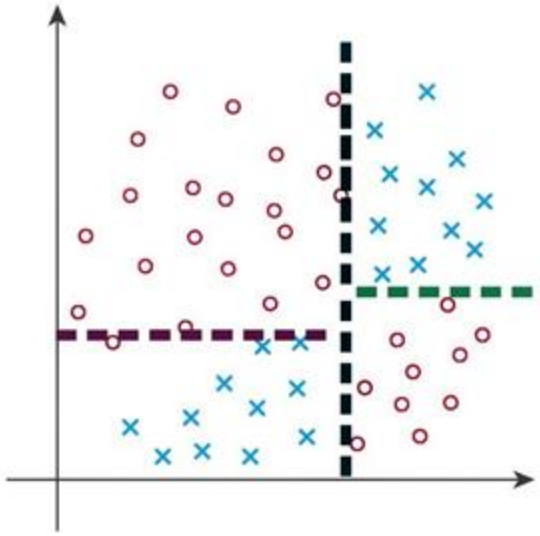
Decision Trees

A decision tree is a graphical representation of all the possible solutions to a decision based on a certain conditions.

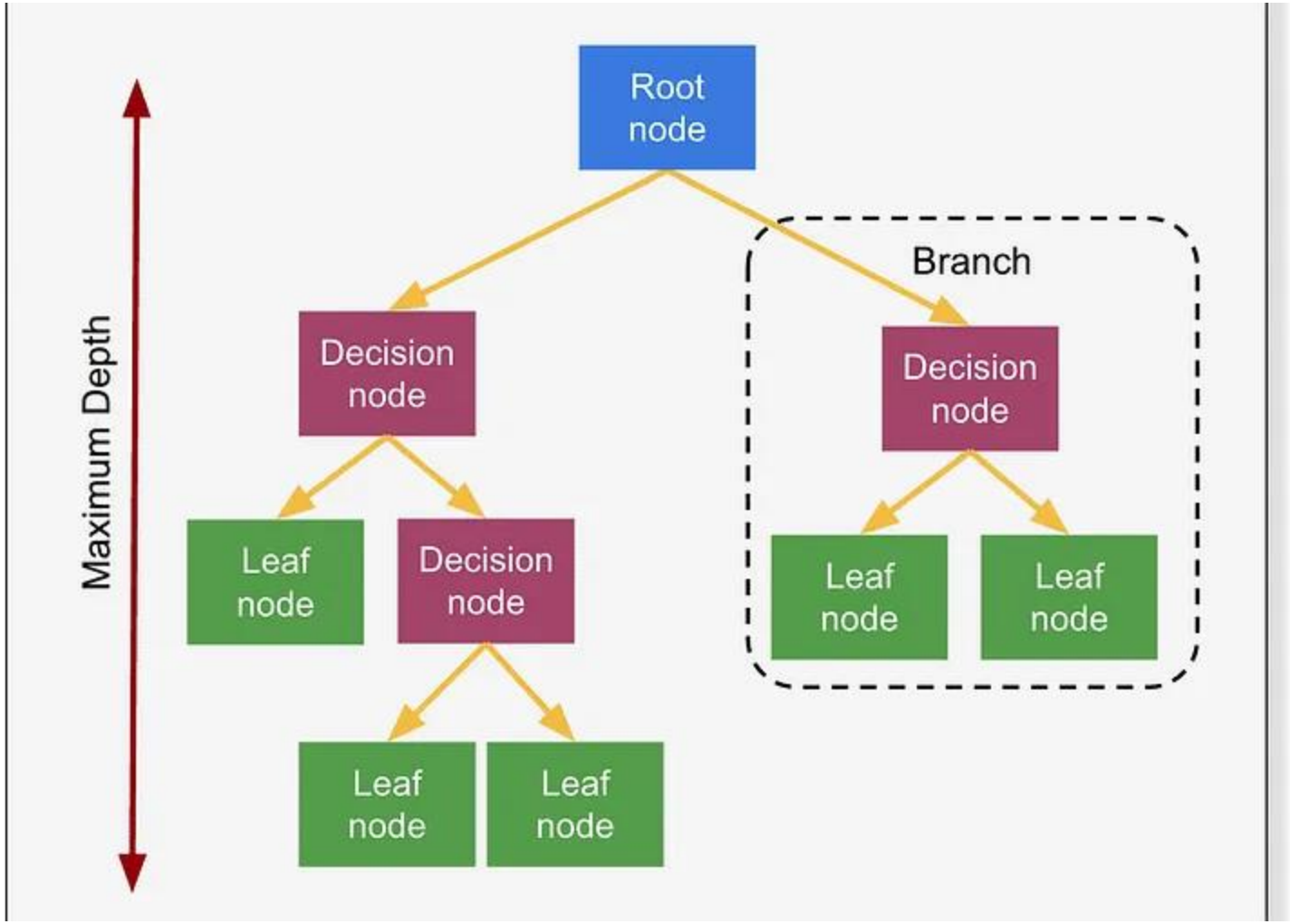
A sequence of questions.



It’s just nested if-else rules.



Decision Tree Terminology



Decision Tree Terminology

- **Root Node:**
  - Represents the entire dataset
- **Decision Nodes:**
  - The nodes where the dataset is further split based on a certain feature.
- **Leaf Nodes:**
  - They represent the final output or the decision of the model.
  - They do not split further and contain the outcome of the decision path.
- **Branch:**
  - The connection between nodes, representing the outcome of a test.
- **Parent Node**
  - Any node that splits into one or more branches
- **Child Node**
  - - Created from a parent node after a split

Algorithm for Decision Tree Induction

- Basic algorithm (a greedy algorithm)
  - Tree is constructed in a top-down recursive divide-and-conquer manner
  - At start, all the training examples are at the root
  - Attributes are categorical (if continuous-valued, they are discretized in advance)
  - Examples are partitioned recursively based on selected attributes
  - Test attributes are selected on the basis of a heuristic or statistical measure (e.g., information gain)
- Conditions for stopping partitioning
  - All samples for a given node belong to the same class
  - There are no remaining attributes for further partitioning –majority voting is employed for classifying the leaf
  - There are no samples left

Example

Goal: Predict whether someone will like go to **:**.

Weekend	Weather	Parents	Money	Decision
W1	Sunny	Yes	Rich	Cinema
W2	Sunny	No	Rich	Tennis
W3	Windy	Yes	Rich	Cinema
W4	Rainy	Yes	Poor	Cinema
W5	Rainy	No	Rich	Stay In
W6	Rainy	Yes	Poor	Cinema
W7	Windy	No	Poor	Cinema
W8	Windy	No	Rich	Shopping
W9	Windy	Yes	Rich	Cinema
W10	Sunny	No	Rich	Tennis

There are 4 possible outputs:

Cinema, Tennis,
Stay In, Shopping.

6 instances of Cinema
2 instances of Tennis
1 instance of Stay In
1 instance of Shopping

What Is Impurity?

Compute the **Gini Index** for the overall collection of training samples:

- A node is **pure** if everyone in it has the same label.
- A node is **impure** if it mixes Yes and No

❖ **Gini impurity:**

$$G = 1 - \sum_{i=1}^{n} (p_i)^2$$

$$Gini = 1 - [(\frac{6}{10})^2 + (\frac{2}{10})^2 + (\frac{1}{10})^2 + (\frac{1}{10})^2] = 0.58$$

- 0 means perfect purity
- 0.5 means maximum impurity (for binary classification).
- The higher the Entropy (or Gini Index), the more random (mixed) the data is

Compute Root Impurity

Computation of Gini index for money attribute:

Has two possible values: Rich (7 Instances) , Poor (3 Instances)

For Money=Poor ⟹ 3 “Cinema”

$$G(s) = 1 - [(\frac{3}{3})^2] = 0$$

For Money=Rich ⟹ 3 “Cinema” , 2 “Tennis”, 1 “Stay In”,1 “Shopping”

$$G(s) = 1 - [(\frac{3}{7})^2 + (\frac{2}{7})^2 + (\frac{1}{7})^2 + (\frac{1}{7})^2] = 0.694$$

Weighted Average Gini ⟹ $= \left(\frac{N_{left}}{N_{total}}\right) * G_{left} + \left(\frac{N_{right}}{N_{total}}\right) * G_{right}$

Weighted Average Gini (Money) $= \frac{3}{10} * 0 + \frac{7}{10} * 0.694 = 0.486$

- Same For Parents and Weather

Weather Has Three possible values: Sunny (3 Instances) , Windy(3 Instances), Rainy(4 Instances)

For Weather = Sunny ⟹ 1 “Cinema” and 2 “Tennis”

$$G(s) = 1 - [(\frac{2}{3})^2 + (\frac{1}{3})^2] = 0.444$$

For Weather=Windy ⟹ 3 “Cinema” , 1 “Shopping”

$$G(s) = 1 - [(\frac{3}{4})^2 + (\frac{1}{4})^2] = 0.375$$

For Weather=Rainy ⟹ 2 “Cinema” , 1 “Stay In”

$$G(s) = 1 - [(\frac{2}{3})^2 + (\frac{1}{3})^2] = 0.444$$

- Weighted Average Gini (Weather) $= \frac{3}{10} * 0.444 + \frac{4}{10} * 0.375 + \frac{3}{10} * 0.444 = 0.416$

- For Money - Gini index $= \frac{3}{10} * 0 + \frac{7}{10} * 0.694 = 0.486$
- For Weather - Gini index $= \frac{3}{10} * 0.444 + \frac{4}{10} * 0.375 + \frac{3}{10} * 0.444 = 0.416$
- For Parents - Gini index $= \frac{5}{10} * 0 + \frac{5}{10} * 0.72 = 0.36$ ✓

Parents is selected as it has the smallest Gini Index

Parents

Yes

Weekend	Weather	Parents	Money	Decision
W1	Sunny	Yes	Rich	Cinema
W3	Windy	Yes	Rich	Cinema
W4	Rainy	Yes	Poor	Cinema
W6	Rainy	Yes	Poor	Cinema
W9	Windy	Yes	Rich	Cinema

No

Weekend	Weather	Parents	Money	Decision
W2	Sunny	No	Rich	Tennis
W5	Windy	No	Rich	Stay In
W7	Rainy	No	Poor	Cinema
W8	Rainy	No	Rich	Shopping
W10	Windy	No	Rich	Tennis

No computation is needed here since it is always Cinema

Computation for Gini Index for Parents=No | Weather Attribute

- Weather =Sunny, 2 Instances both are "Tennis"

$$G(s) = 1 - [(\frac{2}{2})^2] = 0$$

- Weather =Rainy, 1 "Stay In"

$$G(s) = 1 - [(\frac{1}{1})^2] = 0$$

- Weather =Windy, 1 "Cinema" , 1 "Shopping"

$$G(s) = 1 - [(\frac{1}{2})^2 + (\frac{1}{2})^2] = 0.5$$

- Weighted Average Gini (Parents=0|Weather) $= \frac{2}{5} * 0 + \frac{1}{5} * 0 + \frac{2}{5} * 0.5 = 0.2$

Computation for Gini Index for Parents=No | Money Attribute

- Money = Rich, 2 "Tennis" , 1 "Stay In", 1 "Shopping"

$$G(s) = 1 - [(\frac{2}{4})^2 + (\frac{1}{4})^2 + (\frac{1}{4})^2 +] = 0.625$$

- Money = Poor, 1 "Cinema"

$$G(s) = 1 - [(\frac{1}{1})^2] = 0$$

- Weighted Average Gini (Parents=0|Money) $= \frac{4}{5} * 0.625 + \frac{1}{5} * 0 = 0.5$

- For Parents= No|Weather – Gini index = 0.2 ✓
- For Parents= No|Money – Gini index = 0.5

Weather is selected as it has the smallest Gini index

Weekend	Weather	Parents	Money	Decision
W2	Sunny	No	Rich	Tennis
W10	Sunny	No	Rich	Tennis

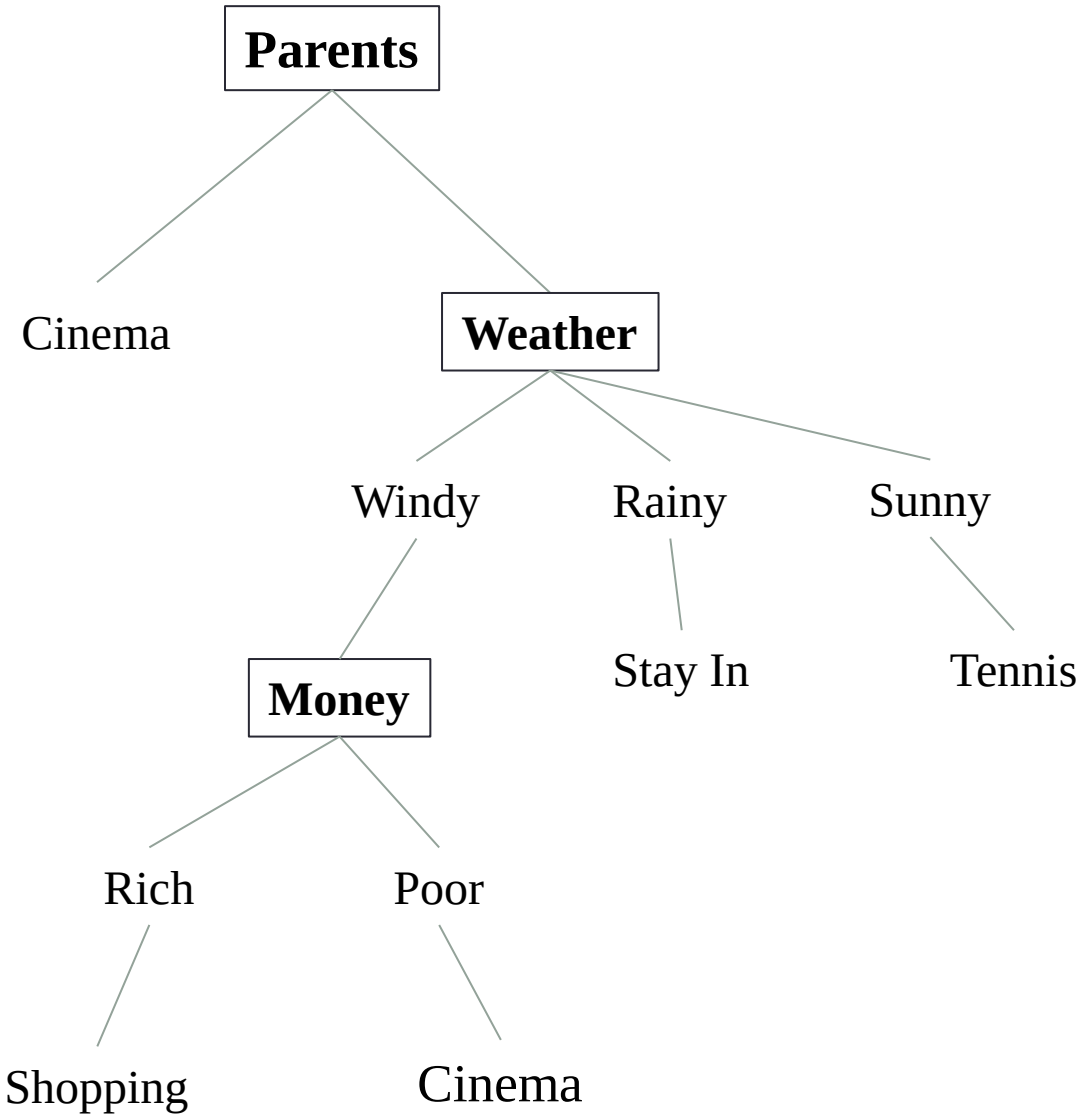
For Parent =No & Weather =Sunny , All instances are Tennis

For Parent =No & Weather =Rainy , All instances are Stay In

Weekend	Weather	Parents	Money	Decision
W5	Rainy	No	Rich	Stay In

Weekend	Weather	Parents	Money	Decision
W7	Windy	No	Poor	Cinema
W8	Windy	No	Rich	Shopping

For Parent =No & Weather =Windy, We need to Split.


Parents
Cinema
Weather
Windy
Rainy
Sunny
Money
Stay In
Tennis
Rich
Poor
Shopping
Cinema

- Other statistical measure is information gain using Entropy

$$\text{Entropy} = -\ \Sigma\ (p_i\ * log_2(p_i))$$

For binary classification, the upper bound of Entropy is 1.

- A tree can grow until it perfectly fits the training data, which often hurts performance on new data.

This introduces:

- Overfitting : Very deep tree → memorizes training data
- Underfitting : Very shallow tree → misses patterns

Tree Pruning

- To fix overfitting we use Pruning

- Pre-pruning (early stopping)

Stop growing the tree when:

  - depth > threshold
  - samples < threshold

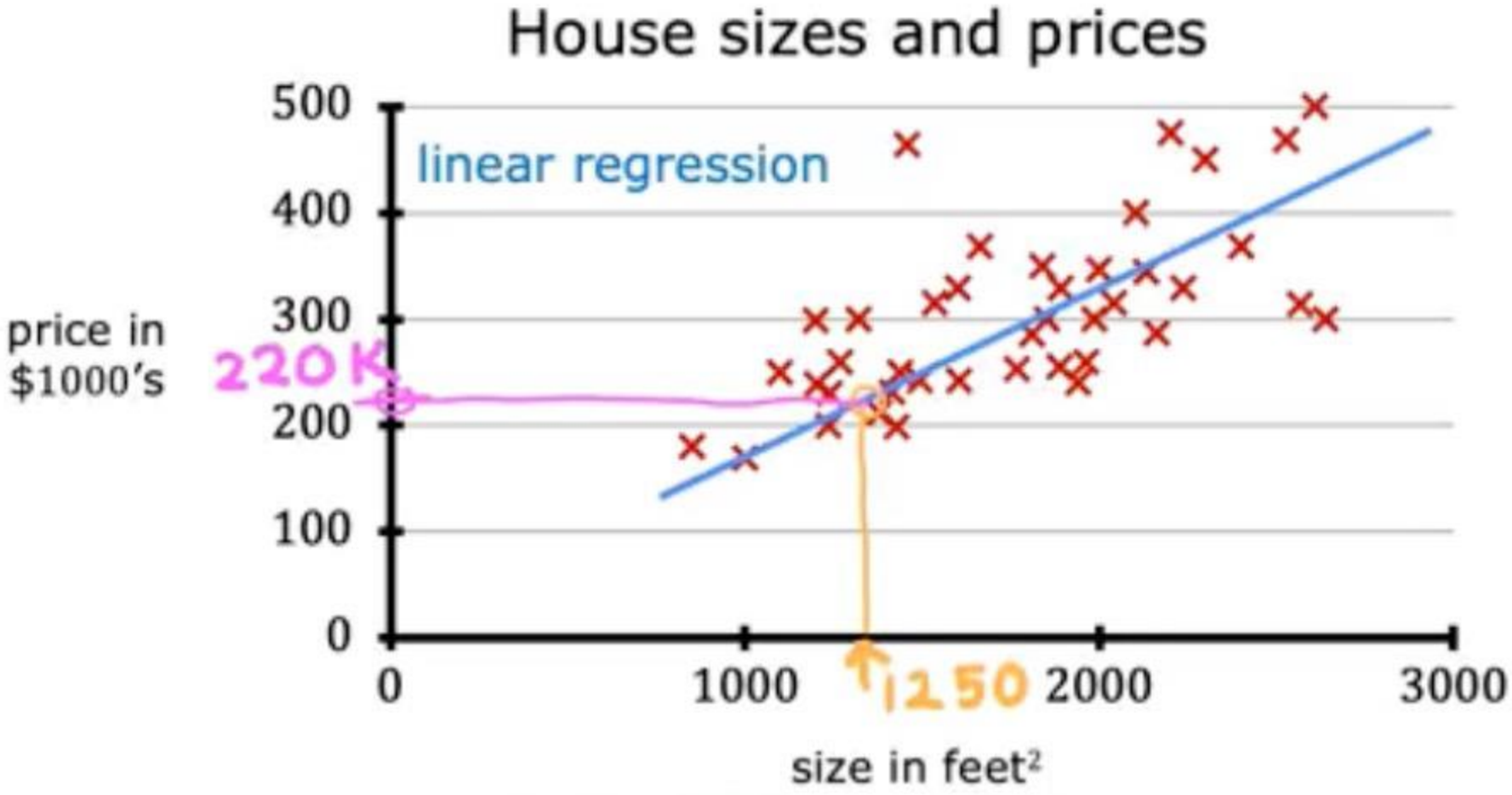
- Post-pruning;

Grow full tree → then remove weak branches.

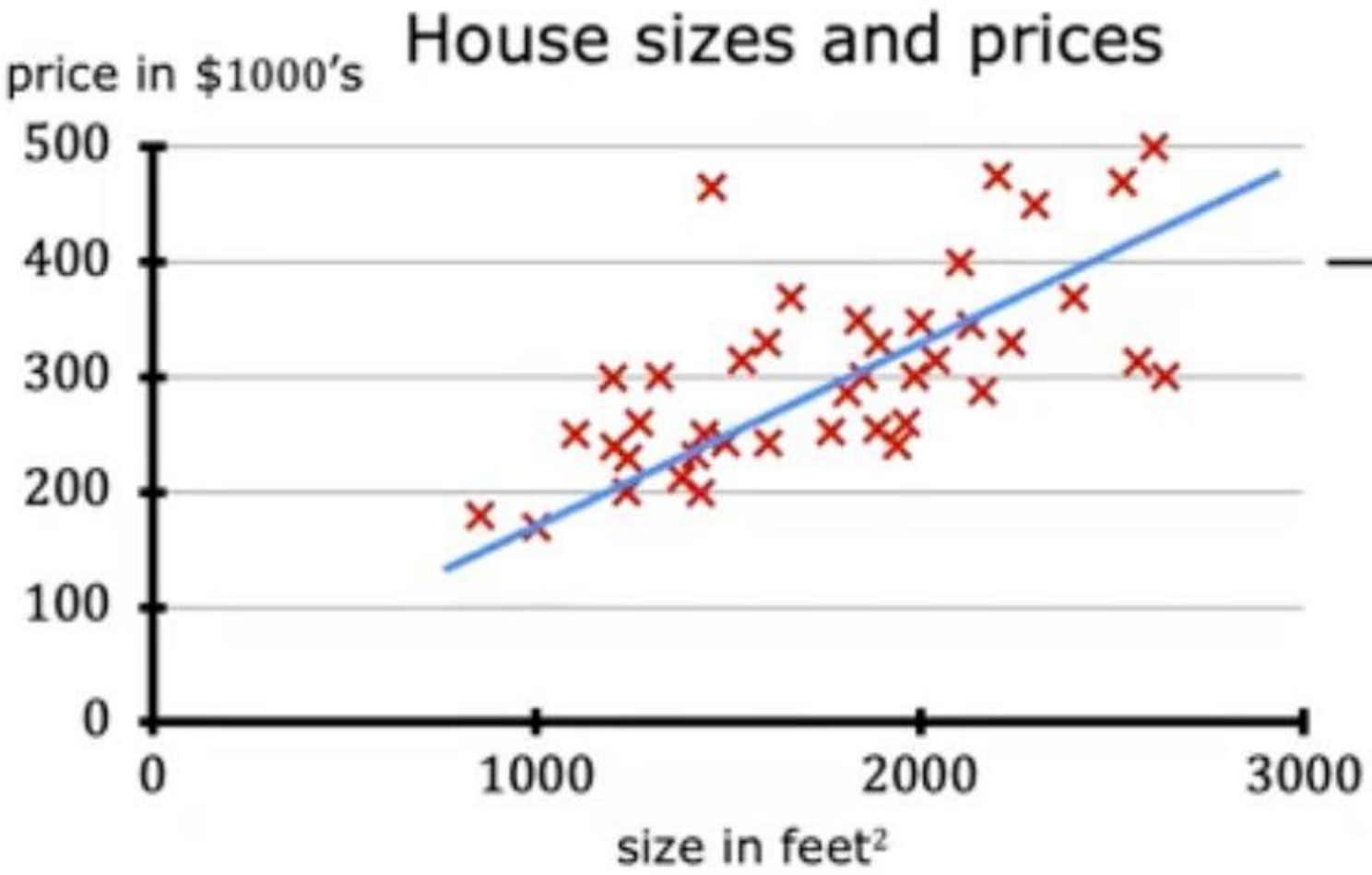
Data Mining

- Linear regression with one variable
  a. Model representation
  b. Cost function
  c. Gradient decent

House sizes and prices
price in
$1000's
500
400
300
200
100
0
0
1000
2000
3000
size in feet²



Supervised learning model



Data table

size in feet²	price in $1000's
2104	400
1416	232
1534	315
852	178
…	…
3210	870